Companion to Bird & de Moor, Algebra of Programming

§8. Thinning Algorithms

§8.1. Thinning

(8.1)

`thin(Q) ≜ ⊇ ∩ (∈ \ (Q° ∈))` — `thin(Q)` takes a set `ys` to a subset `xs` that keeps a `Q`-lower bound of every `a∈ys`.

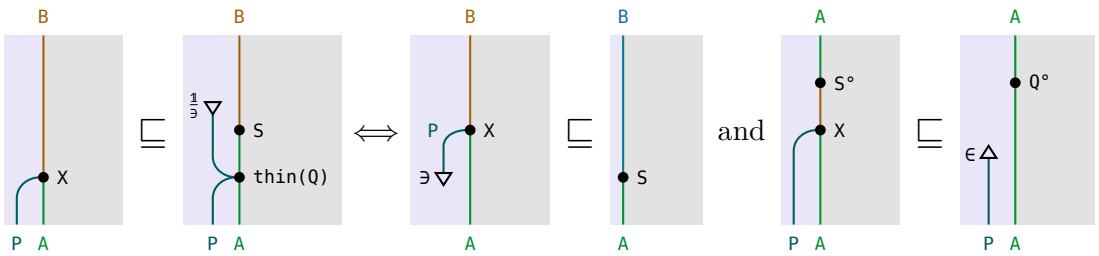
`thin(≤) ys thin(≤) xs⇔xs⊆ys∧(∀a∈ys. ∃b∈xs. b≤a).`

(8.1a)

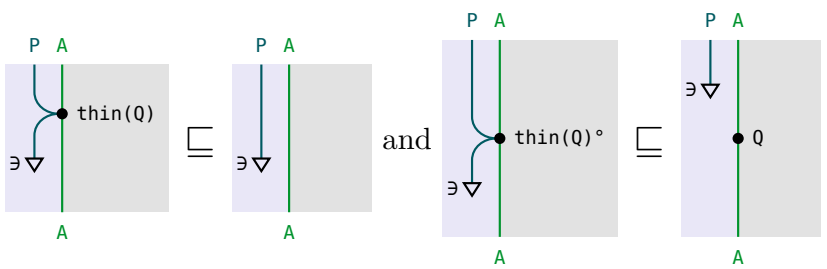
	the law	what it says
1	`X ⊑ S⫐ thin(Q) ⇔ X ∋ ⊑ S ∧ S° X ⊑ Q° ∈`	`S` is the whole algebra: it produces every candidate that `thin(Q)` thins. Everything kept is an `S`-value, and every `S`-value has a `Q`-lower bound among the kept ones
2	`thin(Q) ∋ ⊑ ∋`	everything a thinning keeps was in the set
3	`thin(Q)° ∋ ⊑ ∋ Q`	every element of the set has a `Q`-lower bound among the kept ones
4	`Q ⊑ R ⇒ thin(Q) ⊑ thin(R)`	the fewer pairs `Q` relates, the fewer subsets count as thinnings
5	`𝟙 ⊑ Q ⇒ 𝟙 ⊑ thin(Q)` `Q² ⊑ Q ⇒ thin(Q)² ⊑ thin(Q)`	keeping everything is always a legal thinning
6	`Q ⊑ R; 𝟙 ⊑ Q; R² ⊑ R ⇒ est(R) = thin(Q) est(R) 𝟙 ⊑ Q` `⇒ est(R) ⊑ thin(Q) est(R)`	**thin-introduction**: thinning first cannot lose an `R`-minimum
7	`thin(Q)⫌est(Q) 𝟙⫐` (8.2) — (8.2) `est(Q) 𝟙⫐ ⊑ thin(Q)`	**thin-elimination**: keeping one element is a thinning, but its domain is the sets `est(Q)` is defined on
8	`S⫐ thin(Q)⫌ S⫐ est(R) 𝟙⫐` (8.3), `R∩(S°S)⊑Q` — thin-elimination variant	the usable variant: `R` need only refine `Q` between values `S` gives one argument
9	`P(thin(Q)) union ⊑ union thin(Q)` (8.4) — (8.4) `P(thin(Q)) union ⊑ union thin(Q)`	thinning each member set is a thinning of the union

§8.1.1. `X ⊑ S⫐ thin(Q) ⇔ X ∋ ⊑ S ∧ S° X ⊑ Q° ∈` — the definition of `S⫐ thin(Q)`

(8.1.1a)



(8.1.1b)



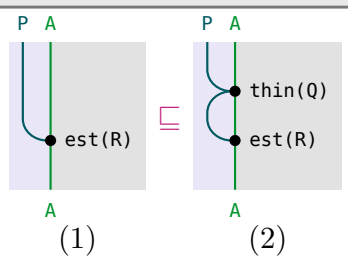
`X ⊑ S⫐ thin(Q) ⇔ X ∋ ⊑ S ∧ S° X ⊑ Q° ∈` at `X≜thin(Q)`, `S≜∋`: the left side is `thin(Q)⊑thin(Q)`, since `∋⫐=∧(∋)=𝟙`

§8.1.2. `Q ⊑ R; 𝟙 ⊑ Q; R² ⊑ R ⇒ est(R) = thin(Q) est(R)` — thinning first, then taking the smallest, is the same as taking the smallest

`𝟙 ⊑ Q` (8.1.2a)

`⇒ est(R) ⊑ thin(Q) est(R)`

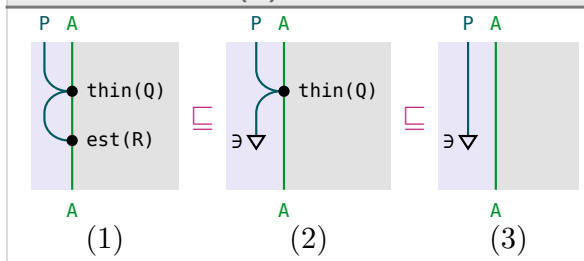
the ⊑ half: keeping everything is a thinning — `𝟙⊑Q`



1	`est(R)`	`𝟙⊑thin(Q)` — `𝟙 ⊑ Q` ⇒ `𝟙 ⊑ thin(Q)` , `Q² ⊑ Q` ⇒ `thin(Q)² ⊑ thin(Q)`
2	`⊑ thin(Q) est(R)`	

`thin(Q) est(R) ⊑ ∋` (8.1.2b)

the ∃ half, first condition of the UP of `est` at `X≜thin(Q) est(R)` — `X ⊑ est(R) ⇔ X ⊑ ∋ ∧ ∈ X ⊑ R°`

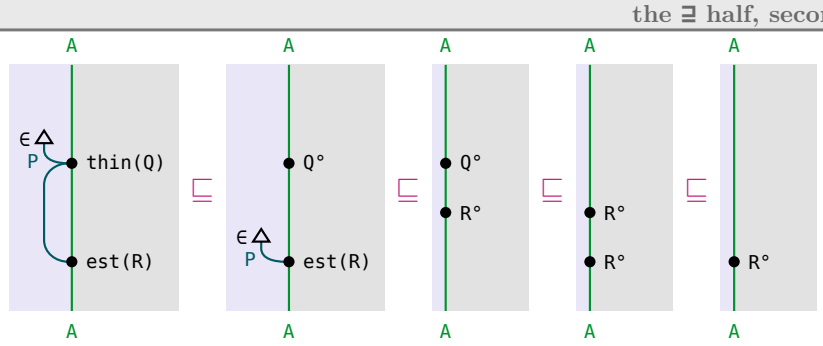


1	`thin(Q) est(R)`	`R ∩ S ⊑ R`
2	`⊑ thin(Q) ∋`	`thin(Q) ∋ ⊑ ∋`
3	`⊑ ∋`	

`Q ⊑ R; R² ⊑ R` (8.1.2c)

`⇒ ∈ thin(Q) est(R) ⊑ R°`

the ∃ half, second condition



1	`∈ thin(Q) est(R)`	`∈ thin(Q) ⊑ Q° ∈`
2	`⊑ Q° ∈ est(R)`	`∈ est(R) ⊑ R°`
3	`⊑ Q° R°`	`Q ⊑ R`
4	`⊑ R°²`	`R² ⊑ R` ⇒ `R°² ⊑ R°`
5	`⊑ R°`	

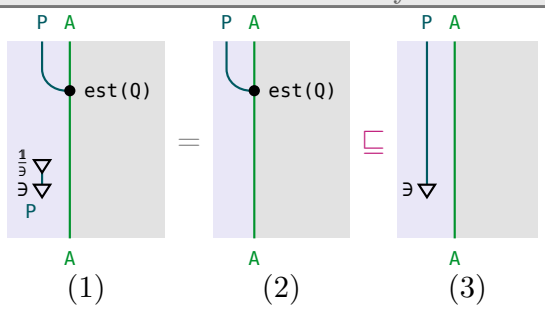
§8.1.3. Properties of thinning

`est(Q)` $\frac{1}{\ni}$ ⊑ `thin(Q)` (8.2)

the singleton holding a `Q`-least member of a set is a thinning of that set

`est(Q)` $\frac{1}{\ni}$ ∋ ⊑ ∋

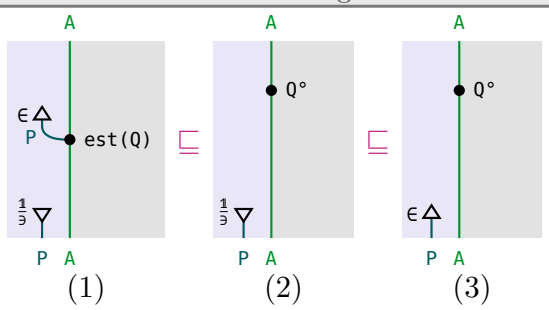
every member of the singleton is a member of the set — ≜ **`thin`**, `∋/∋` half



1	`est(Q)` $\frac{1}{\ni}$ ∋	$\frac{1}{\ni}$ ∋ = 𝟙
2	= `est(Q)`	R ∩ S ⊑ R
3	⊑ ∋	

∈ `est(Q)` $\frac{1}{\ni}$ ⊑ `Q°` ∈

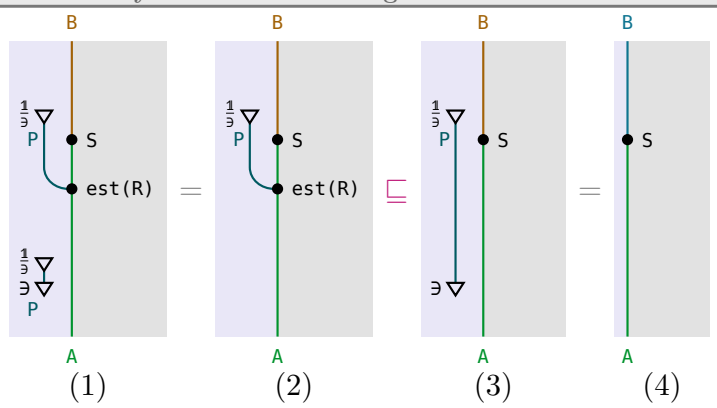
the singleton's member stands in `Q` to every member of the set — ≜ **`thin`**, `∈\(Q°∈)` half



1	∈ `est(Q)` $\frac{1}{\ni}$	∈ `est(Q)` ⊑ `Q°`
2	⊑ `Q°` $\frac{1}{\ni}$	$\frac{1}{\ni}$ ⊑ ∈
3	⊑ `Q°` ∈	

$$R \cap (S^\circ\ S) \sqsubseteq Q \qquad (8.3)$$
$$\Rightarrow \tfrac{S}{\ni}\ est(R)\ \tfrac{1}{\ni} \sqsubseteq \tfrac{S}{\ni}\ thin(Q)$$
$$\tfrac{S}{\ni}\ est(R)\ \tfrac{1}{\ni}\ \ni \sqsubseteq S$$

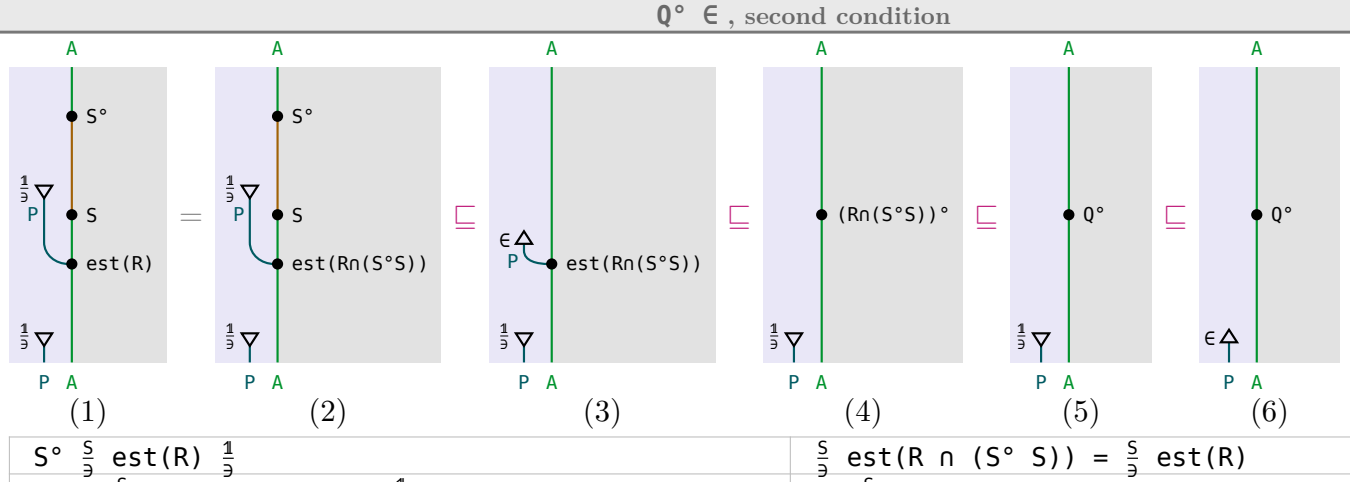
every member of the singleton is a value S returns — $X \sqsubseteq \tfrac{S}{\ni}\ thin(Q) \Leftrightarrow X\ \ni \sqsubseteq S \wedge S^\circ\ X \sqsubseteq Q^\circ \in$, first condition



1	$\tfrac{S}{\ni}\ est(R)\ \tfrac{1}{\ni}\ \ni$	$\tfrac{1}{\ni}\ \ni = \mathbb{1}$
2	$= \tfrac{S}{\ni}\ est(R)$	$R \cap S \sqsubseteq R$
3	$\sqsubseteq \tfrac{S}{\ni}\ \ni$	$\tfrac{S}{\ni}\ \ni = S$
4	$= S$	

$$R \cap (S^\circ\ S) \sqsubseteq Q$$
$$\Rightarrow S^\circ\ \tfrac{S}{\ni}\ est(R)\ \tfrac{1}{\ni} \sqsubseteq Q^\circ \in$$

the singleton's member stands in Q to every value S returns at the same argument — $X \sqsubseteq \tfrac{S}{\ni}\ thin(Q) \Leftrightarrow X\ \ni \sqsubseteq S \wedge S^\circ\ X \sqsubseteq Q^\circ \in$, second condition



1	$S^\circ\ \tfrac{S}{\ni}\ est(R)\ \tfrac{1}{\ni}$	$\tfrac{S}{\ni}\ est(R \cap (S^\circ\ S)) = \tfrac{S}{\ni}\ est(R)$
2	$= S^\circ\ \tfrac{S}{\ni}\ est(R \cap (S^\circ\ S))\ \tfrac{1}{\ni}$	$S^\circ\ \tfrac{S}{\ni} \sqsubseteq \in$
3	$\sqsubseteq \in\ est(R \cap (S^\circ\ S))\ \tfrac{1}{\ni}$	$\in\ est(R) \sqsubseteq R^\circ$
4	$\sqsubseteq (R \cap (S^\circ\ S))^\circ\ \tfrac{1}{\ni}$	$R \cap (S^\circ\ S) \sqsubseteq Q$
5	$\sqsubseteq Q^\circ\ \tfrac{1}{\ni}$	$\tfrac{1}{\ni} \sqsubseteq \in$
6	$\sqsubseteq Q^\circ \in$	

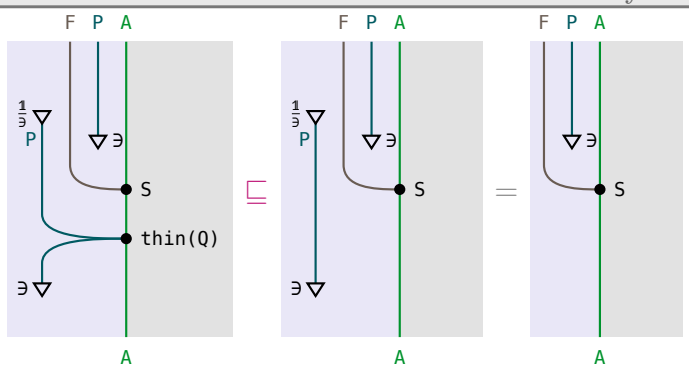
preorder(Q); S Pres Q
⇒ (⦇$\frac{F(∋)\ S}{∋}$ thin(Q)⦈ ⊑ $\frac{⦇S⦈}{∋}$ thin(Q)

Theorem 8.1

thinning at every step of the reduce is a thinning of the whole candidate set

(⦇$\frac{F(∋)\ S}{∋}$ thin(Q)⦈ ∋ ⊑ ⦇S⦈

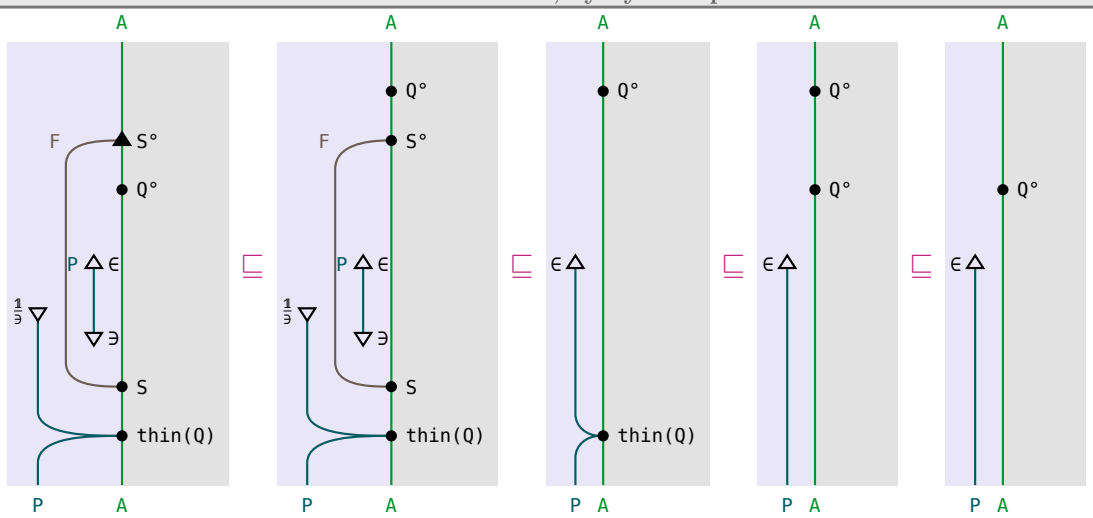
everything the thinning fold keeps is a value of the plain fold — **X ⊑ $\frac{S}{∋}$ thin(Q) ⇔ X ∋ ⊑ S ∧ S° X ⊑ Q° ∈** , first condition, by fusion from the chain below



1	$\frac{F(∋)\ S}{∋}$ thin(Q) ∋	thin(Q) ∋ ⊑ ∋
2	⊑ $\frac{F(∋)\ S}{∋}$ ∋	$\frac{R}{∋}$ ∋ = R
3	= F(∋) S	

preorder(Q); S Pres Q
⇒ ⦇S⦈° (⦇$\frac{F(∋)\ S}{∋}$ thin(Q)⦈ ⊑ Q° ∈

every value of the plain fold is Q-above one the thinning fold keeps — **X ⊑ $\frac{S}{∋}$ thin(Q) ⇔ X ∋ ⊑ S ∧ S° X ⊑ Q° ∈** , second condition, by hylomorphism theorem from the chain below



1	S° F(Q°) F(∈) $\frac{F(∋)\ S}{∋}$ thin(Q)	S Pres Q ⇒ S° F(Q°) ⊑ Q° S°
2	⊑ Q° S° F(∈) $\frac{F(∋)\ S}{∋}$ thin(Q)	S° F(∈) $\frac{F(∋)\ S}{∋}$ ⊑ ∈
3	⊑ Q° ∈ thin(Q)	∈ thin(Q) ⊑ Q° ∈
4	⊑ Q°² ∈	
5	⊑ Q° ∈	

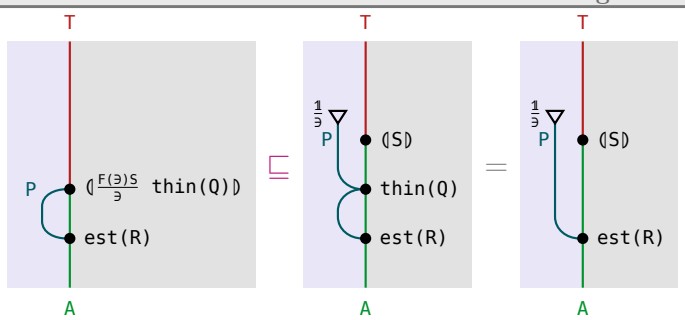
(8.1.3a)

	step	type	example	what it does
1	S:F(A)→A is 8.1's algebra; example F(X)=V+V×X, S=[τ,cons], candidate a=cons(v,p)			
2	S°	A→FA	cons(v,p) ↦ r(v,p)	takes a apart into the input of its last step
3	F(Q°∈)	FA→F(PA)	r(v,p) ↦ r(v,ps), some p'∈ps with p' Q p	replaces each recursive component by a set holding a Q-no-worse one; v stays
4	$\frac{F(∋)S}{∋}$	F(PA)→PA	r(v,ps) ↦ {cons(v,t) | t∈ps}	picks one element from each set, combines by S, collects every result
5	thin(Q)	PA→PA	{cons(v,t) | t∈ps} ↦ ys	keeps a subset; every dropped element is Q-beaten by a kept one
6	⊑ Q°∈	A→PA	ys holds y with y Q cons(v,p') Q cons(v,p) = a	the thinned set still holds an element Q-no-worse than a (S monotonic on Q, Q transitive)

Q ⊑ R; preorder(Q); preorder(R); S Pres Q
⇒ (⦇F(∋) S / ∋ thin(Q)⦈ est(R) ⊑ ⦇S⦈ / ∋ est(R)

Corollary 8.1

the thinning fold refines the optimisation problem itself



1	⦇F(∋) S / ∋ thin(Q)⦈ est(R)	Theorem 8.1 preorder(Q); S Pres Q ⇒ ⦇F(∋) S / ∋ thin(Q)⦈ ⊑ ⦇S⦈ / ∋ thin(Q)
2	⊑ ⦇S⦈ / ∋ thin(Q) est(R)	
3	= ⦇S⦈ / ∋ est(R)	

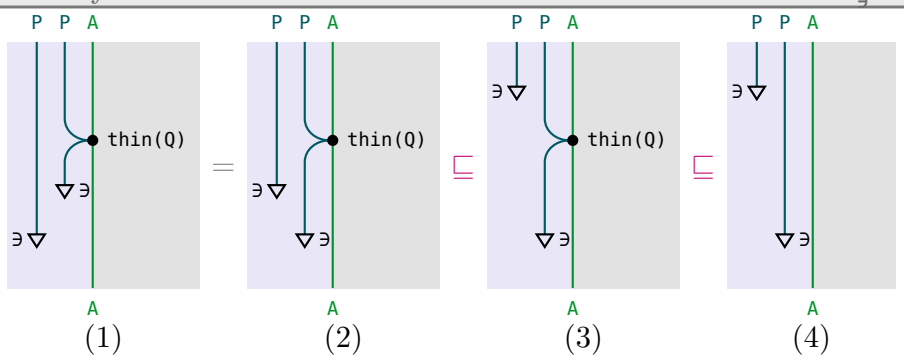
P(thin(Q)) union ⊑ union thin(Q)

(8.4)

thinning each member set, then taking the union, is a thinning of the union

P(thin(Q)) union ∋ ⊑ ∋ ∋

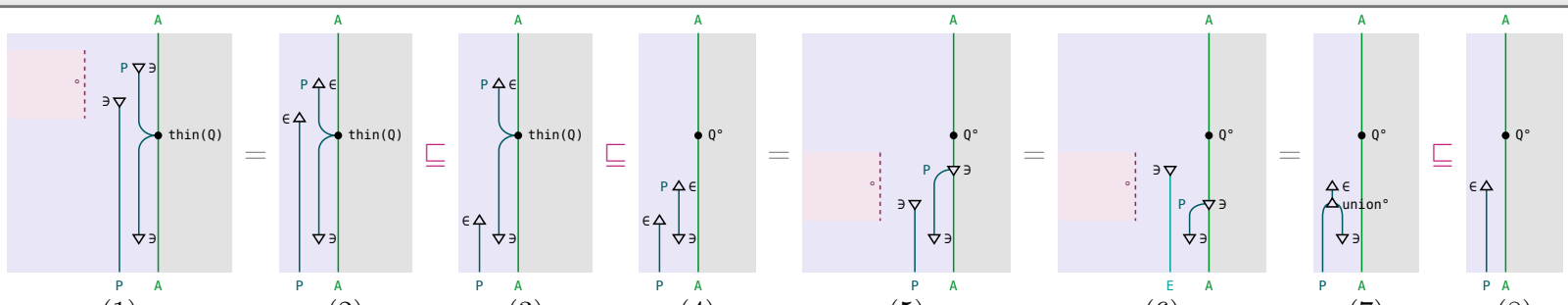
every member of the result is a member of a member set — X ⊑ S / ∋ thin(Q) ⇔ X ∋ ⊑ S ∧ S° X ⊑ Q° ∈ , first condition



1	P(thin(Q)) union ∋	union ∋ = ∋ ∋
2	= P(thin(Q)) ∋ ∋	P(R) ∋ ⊑ ∋ R
3	⊑ ∋ thin(Q) ∋	thin(Q) ∋ ⊑ ∋
4	⊑ ∋ ∋	

(∋ ∋)° P(thin(Q)) union ⊑ Q° ∈

every member of a member set has a Q-lower bound in the result — X ⊑ S / ∋ thin(Q) ⇔ X ∋ ⊑ S ∧ S° X ⊑ Q° ∈ , second condition



1	(∋ ∋)° P(thin(Q)) union	(R S)° = S° R°
2	= ∈ ∈ P(thin(Q)) union	∈ P(R) ⊑ R ∈
3	⊑ ∈ thin(Q) ∈ union	∈ thin(Q) ⊑ Q° ∈
4	⊑ Q° ∈ ∈ union	(R S)° = S° R°
5	= Q° (∋ ∋)° union	union ∋ = ∋ ∋
6	= Q° (union ∋)° union	(union S)° = S° union°
7	= Q° ∈ union° union	union = E(∋)
8	⊑ Q° ∈	

§8.2. Paths in a layered network

(8.2a)

	definition	type	meaning
1	`F(B,X) = B + (B × X)`	$\mathcal{A}\times\mathcal{A}\to\mathcal{A}$	a path is either one vertex a, or a vertex a followed by a path xs
2	`α = [τ,cons]`	`F(A,LA)→LA`	build a path: wrap one vertex as [a], or put a in front of xs
3	`wrapz ≜ ⟨τ,zero⟩`	`A→LA×ℕ`	a path of one vertex, no edge, cost 0
4	`(a,(xs,n)) ↦ ([a]⧺xs,wt(a,head(xs)) + n)`	`A×(LA×ℕ)→LA×ℕ`	put a in front of the path; the cost adds the weight of the new edge a→head(xs)
5	`cost ≜ ⦇[wrapz,consw]⦈ π₂`	`LA→ℕ`	the cost of $[a_0,\dots,a_n]$ is $wt(a_0,a_1)+\dots+wt(a_{n-1},a_n)$
6	`R ≜ cost ≤ cost°`	`LA→LA`	xs is cheaper than ys
7	`⦇F(∋,𝟙) α⦈ = L(∋)`	`LA→LB`	L(∋) relates a list of layers to every path that chooses one vertex from each layer
8	`minpath ⊑ L(∋)/∋ est(R)`	`L(PA)→LA`	the input $[x_0,\dots,x_n]$: L(PA) is a list of layers, each a set of vertices; a path chooses one vertex from each layer, and minpath returns a cheapest one
9	`Q ≜ R ∩ (head head°)`	`LA→LA`	like R, and the two paths also start at the same vertex: head(xs) = head(ys)
10	`a ↦ l(a)`	`B→B+C`	mark a value as the left alternative
11	`b ↦ r(b)`	`C→B+C`	mark a value as the right alternative
12	`cpl ≜ cp(𝟙 × A)`	`PB×A→P(B×A)`	pair every element of the set with the value beside it
13	`cpr ≜ cp(A × 𝟙)`	`A×PB→P(A×B)`	pair the value beside the set with every element of the set
14	`F(∋,𝟙)/∋ = [P(l),cpl P(r)]`	`F(PB,X)→PF(B,X)`	cpl transpose: from layer x and a set of paths ps, choose a vertex a ∈ x
15	`F(𝟙,∋)/∋ = [𝟙/∋ P(l),cpr P(r)]`	`F(B,PX)→PF(B,X)`	cpr transpose: from a vertex a and a set of paths ps, choose a tail xs ∈ ps
16	`step ≜ cpr P(cons) est(R)`	`A×P(LA)→LA`	put a in front of every path in ps and keep a cheapest one

(8.2b)

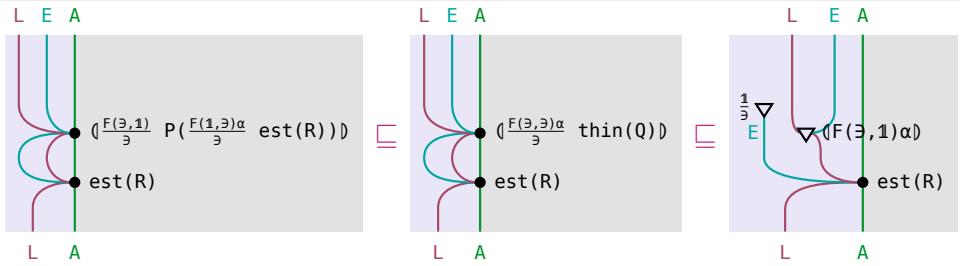
	the law	what it says
1	`F(∋,Q)α⊑F(∋,𝟙)αQ`	`F(∋,𝟙)α` is monotonic on `Q`; on `R` it is not, since the next edge can cost arbitrarily much
2	`S head⊑[𝟙,π₁]` `S≜F(𝟙,∋)α`	`S head` is simple, which gives `R∩(S°S)⊑Q`: between two paths `S` builds from one argument, equal cost and equal head already means `Q`

§8.2.1. Derivation

$$(\!|\tfrac{F(\exists,1)}{\exists}\ P(\tfrac{F(1,\exists)\ \alpha}{\exists}\ \mathtt{est(R)})|\!)\ \mathtt{est(R)} \sqsubseteq \tfrac{(\!|F(\exists,1)\ \alpha|\!)}{\exists}\ \mathtt{est(R)} \qquad (8.2.1a)$$

a least-cost path in a layered network, as a fold over the layers

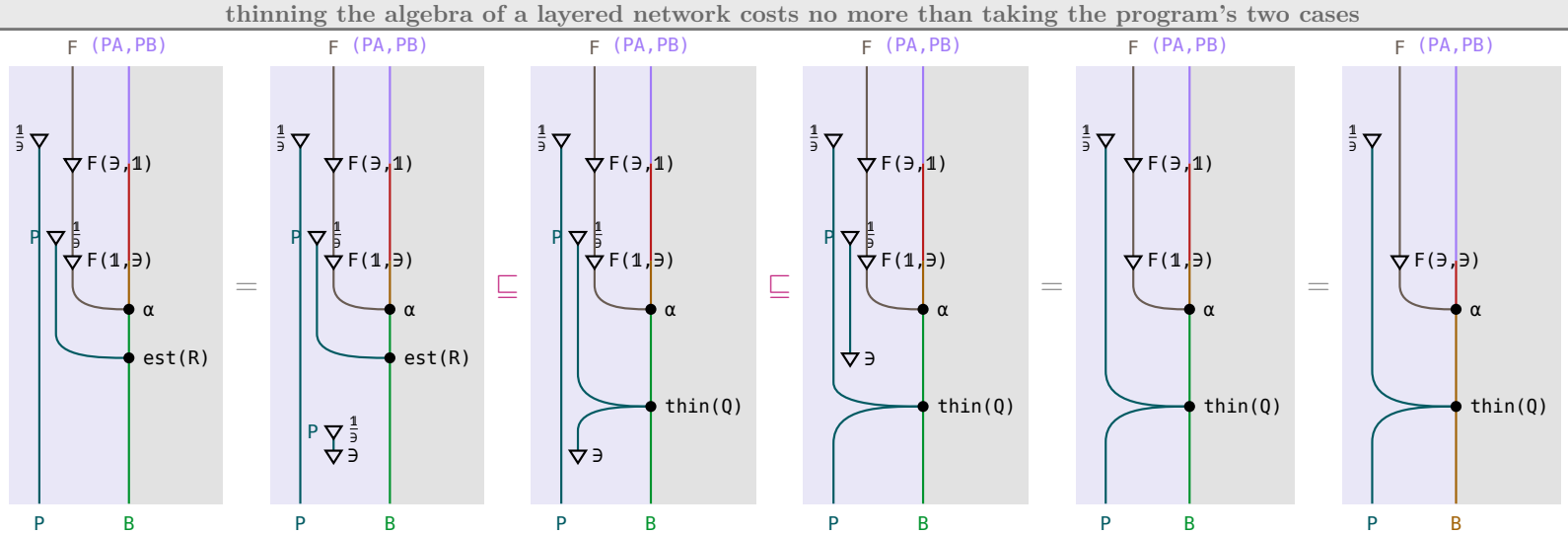
- Why it is needed: `est(R)` keeps one cheapest `w` of the set `Λ(S)(x)`; `thin(Q)` needs every dropped `z` to be `Q`-beaten by a kept one. For `{w}` alone to be a thinning we need `w Q z` for every `z` in `Λ(S)(x)`.
- We have `w R z` (`w` is cheapest) and `w (S°S) z` (both are built by `S` from the same argument `x`); the hypothesis turns the two into `w Q z`: among candidates from one argument, cheaper already means `Q`-better.



1	$(\!\|\tfrac{F(\exists,1)}{\exists}\ P(\tfrac{F(1,\exists)\ \alpha}{\exists}\ \mathtt{est(R)})\|\!)\ \mathtt{est(R)}$	$\tfrac{F(\exists,1)}{\exists}\ P(\tfrac{F(1,\exists)\ \alpha}{\exists}\ \mathtt{est(R)}) \sqsubseteq \tfrac{F(\exists,\exists)\ \alpha}{\exists}\ \mathtt{thin(Q)}$
2	$\sqsubseteq (\!\|\tfrac{F(\exists,\exists)\ \alpha}{\exists}\ \mathtt{thin(Q)}\|\!)\ \mathtt{est(R)}$	$(\!\|\tfrac{F(\exists,\exists)\ \alpha}{\exists}\ \mathtt{thin(Q)}\|\!)\ \mathtt{est(R)} \sqsubseteq \tfrac{(\!\|F(\exists,1)\ \alpha\|\!)}{\exists}\ \mathtt{est(R)}$
3	$\sqsubseteq \tfrac{(\!\|F(\exists,1)\ \alpha\|\!)}{\exists}\ \mathtt{est(R)}$	

$$\mathtt{S} \triangleq \mathtt{F(1,\exists)\ \alpha}\ ,\ \mathtt{R} \cap (\mathtt{S}^\circ\ \mathtt{S}) \sqsubseteq \mathtt{Q} \Rightarrow \tfrac{F(\exists,1)}{\exists}\ \mathtt{P}(\tfrac{F(1,\exists)\ \alpha}{\exists}\ \mathtt{est(R)}) \sqsubseteq \tfrac{F(\exists,\exists)\ \alpha}{\exists}\ \mathtt{thin(Q)} \qquad (8.2.1b)$$

thinning the algebra of a layered network costs no more than taking the program's two cases



1	$\tfrac{F(\exists,1)}{\exists}\ P(\tfrac{F(1,\exists)\ \alpha}{\exists}\ \mathtt{est(R)})$	$P(\tfrac{1}{\exists})\ E(\exists) = \mathbb{1}$
2	$= \tfrac{F(\exists,1)}{\exists}\ P(\tfrac{F(1,\exists)\ \alpha}{\exists}\ \mathtt{est(R)}\ \tfrac{1}{\exists})\ E(\exists)$	(8.3) $\mathtt{R} \cap (\mathtt{S}^\circ\ \mathtt{S}) \sqsubseteq \mathtt{Q} \Rightarrow \tfrac{S}{\exists}\ \mathtt{est(R)}\ \tfrac{1}{\exists} \sqsubseteq \tfrac{S}{\exists}\ \mathtt{thin(Q)}$
3	$\sqsubseteq \tfrac{F(\exists,1)}{\exists}\ P(\tfrac{F(1,\exists)\ \alpha}{\exists})\ P(\mathtt{thin(Q)})\ E(\exists)$	(8.4) $P(\mathtt{thin(Q)})\ \mathtt{union} \sqsubseteq \mathtt{union}\ \mathtt{thin(Q)}$
4	$\sqsubseteq \tfrac{F(\exists,1)}{\exists}\ P(\tfrac{F(1,\exists)\ \alpha}{\exists})\ E(\exists)\ \mathtt{thin(Q)}$	$\tfrac{\bigvee S}{\exists} = \tfrac{\bigvee}{\exists}\ P(\tfrac{S}{\exists})\ \mathtt{union}$
5	$= \tfrac{F(\exists,1)\ F(1,\exists)\ \alpha}{\exists}\ \mathtt{thin(Q)}$	$\mathtt{F(R,\mathbb{1})\ F(\mathbb{1},S) = F(R,S)}$
6	$= \tfrac{F(\exists,\exists)\ \alpha}{\exists}\ \mathtt{thin(Q)}$	

$= \mathtt{cp(F)\ P(\alpha)\ est(R) = [\tau,step]}$

at the layered network ($\mathtt{F(A,X)=A+A\times X}$, $\mathtt{B}$ the paths $\mathtt{V}^+$, $\mathtt{\alpha=[\tau,cons]}$): $\tfrac{F(1,\exists)}{\exists}\ \mathtt{P(\alpha)\ est(R)=[\tau,step]}$ — $\mathtt{step} \triangleq \mathtt{cpr\ P(cons)\ est(R)}$

$= \tfrac{F(\exists,1)}{\exists}\ \mathtt{=\mathbb{1}+cpl}$ — $\tfrac{F(\exists,\mathbb{1})}{\exists}\ \mathtt{= [P(l),cpl\ P(r)]}$

(8.2.1c)

```
F(A,X)   =  A   +   A  ×  X
            |       |     |
            R       R     S
            ↓       ↓     ↓
F(A',X') =  A'  +   A' ×  X'
```

`u F(R,S) v` holds iff one of:

- `u`, `v` are both `inl` and `d R d'`: the leaf moves by `R`;
- `u`, `v` are both `inr` and $\mathtt{p_1\ R\ q_1} \wedge \mathtt{p_2\ S\ q_2}$: the vertex moves by `R`, the rest by `S`.
- An `inl` never relates to an `inr`: $\mathtt{R+S} \triangleq \mathtt{[R\ inl,\ S\ inr]}$ returns to the summand it came from, and $\mathtt{F(\mathbb{1})=\mathbb{1}}$ allows no crossing.
- $\mathtt{F(\exists,\exists)}$ relates `inr({a,b},{p,q})` to `inr(a,p)`, `inr(a,q)`, `inr(b,p)`, `inr(b,q)`, and `inl({a,b})` to `inl(a)`, `inl(b)`.

§8.3. Implementing thin

(8.3a)

	definition	type	meaning
1	`(setify ∋)(xs,x) ⇔ x ∈ xs`	`[A]→PA`	the set of a list's elements
2	`cup ≜ (π₁ ∋) ∪ (π₂ ∋) / ∋`	`PA×PA→PA`	the union of two sets
3	`cp(F) ≜ F(∋) / ∋`	`F(PA)→PFA`	every `F`-shaped value whose parts are chosen from the given sets

4	`cpLFn(zer,x) = []`
	`cpLFn(PolyC.one,u) = [u, ()]`
	`cpLFn(const(a),c) = [c]`
	`cpLFn(arg₁,a) = [a]`
	`cpLFn(arg₂,xs) = xs`
	`cpLFn(l ⊕ a,l(u)) = inl′(cpLFn(l,u))`
	`cpLFn(a ⊕ r,r(v)) = inr′(cpLFn(r,v))`
	`cpLFn(l ⊗ r,(u,v)) = cppFn(cpLFn(l,u),cpLFn(r,v))`
		`relator(list(X))→`
		`list(relator(X))`
			by the shape of `F`: a constant or `arg₁` is a one-element list, `arg₂` the list itself, a sum lists its summand, a product pairs every element of one list with every element of the other

5	`linear`	`P(PolyC(ι))`	no product in `F` has the argument on both sides
6	`sort(⩽) ≜ setify° ordered(⩽)`	`PA→LA`	read a set back as one of its `⩽`-ordered listings
7	`bump(Q)`	`A×[A]→[A]`	put `a` in front of a thinned list, dropping whichever of `a` and the old head the other beats
8	`thinL(Q) ≜ (setify thin(Q) setify°) ∩ subseq`	`[A]→[A]`	only drop elements, and on the set of the elements be a thinning by `Q`
9	`T ⊑ thinL(Q) ⇔ T ⊑ subseq ∧ T setify ⊑ setify thin(Q)`		what a list implementation of `thin(Q)` must do: only drop elements, and thin the set
10	`preorder(Q) ⇒ ◖[[],bump(Q)]◗ ⊑ thinL(Q)`		an implementation: thin a list in one pass, bumping each element into the thinned rest
11	`minL(Q) ≜ setify est(Q)`	`[A]→A`	a `Q`-least element of the list

```
linear(zer) = true
linear(one) = true
linear(const(a)) = true
linear(arg₁) = true
linear(arg₂) = true
linear(l ⊕ r) = linear(l) ∧ linear(r)
linear(l ⊗ r) = linear(l) ∧ linear(r) ∧ (¬hasArg₂(l) ∨ ¬hasArg₂(r))

(a,[]) ↦ [a]
bump(Q)((a,[b]⧺xs),ys) ⇔ Q(a,b) ∧ ys = [a]⧺xs ∨ Q(b,a) ∧ ys = [b]⧺xs ∨ ¬Q(a,b) ∧ ¬Q(b,a) ∧ ys = [a]⧺[b]⧺xs
```

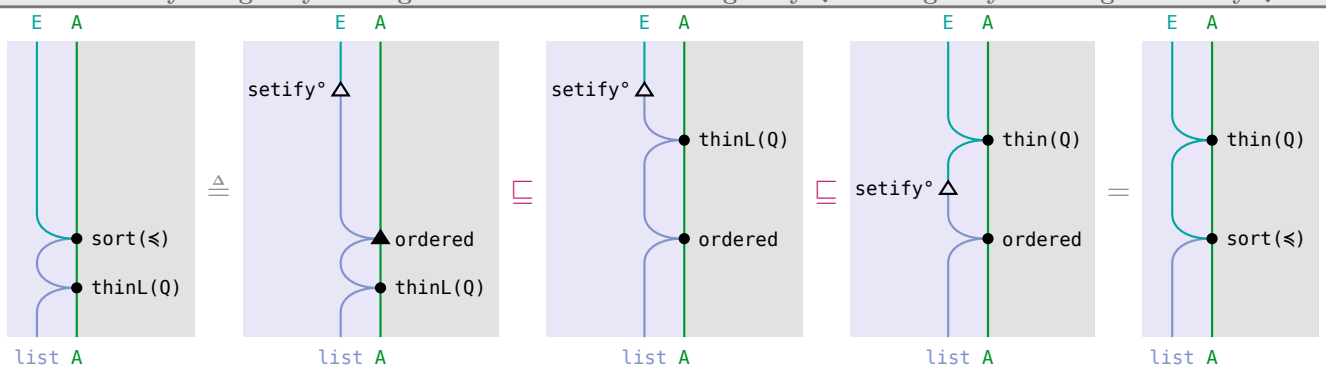
§8.3.1. Sorting sets

(8.3.1a)

	the law	what it says
1	`preorder(Q); connected(Q) ⇒ bumpFold(Q)([a]⧺xs,ys) ⇔ (∃w. minL(Q)([a]⧺xs,w) ∧ ys = [w])` (8.5)	what thinning should come to when it can: one element
2	`sort(⩽) thinL(Q) ⊑ thin(Q) sort(⩽)` (8.6) — (8.6) `sort(⩽) thinL(Q) ⊑ thin(Q) sort(⩽)`	the thinning may run after the sort
3	`sort(⩽) minL(R) ⊑ est(R)` (8.7)	a minimum of the sorted list is a minimum of the set
4	`sort(f ⩽ f°) f′ ⊑ P(f) sort(⩽)` (8.8)	shunt a function through a sort
5	`Coreflexive(p) ⇒ sort(⩽) filter(p) ⊑ E(p) sort(⩽)` (8.9) `p : A → A`	filtering a sorted list sorts the restricted set
6	`preorder(⩽); connected(⩽) ⇒ (sort(⩽) × sort(⩽)) merge(⩽) ⊑ cup sort(⩽)` (8.10)	merging two sorted lists sorts their union
7	`F(sort(⩽)) cpL ⊑ cp(F) sort(F(⩽))` (8.11), `F(A,X)=1+A×X`	`cpL` is the list implementation of the cartesian product `cp(F)`
8	`linear(F) ⇒ relator(sort(⩽)) cpL(F,A,X) ⊑ cp(relator(F,A)) sort(relator(⩽))` (8.11), `F` polynomial and linear	the same for every linear polynomial `F`

`sort(⩽) thinL(Q) ⊑ thin(Q) sort(⩽)` (8.6)

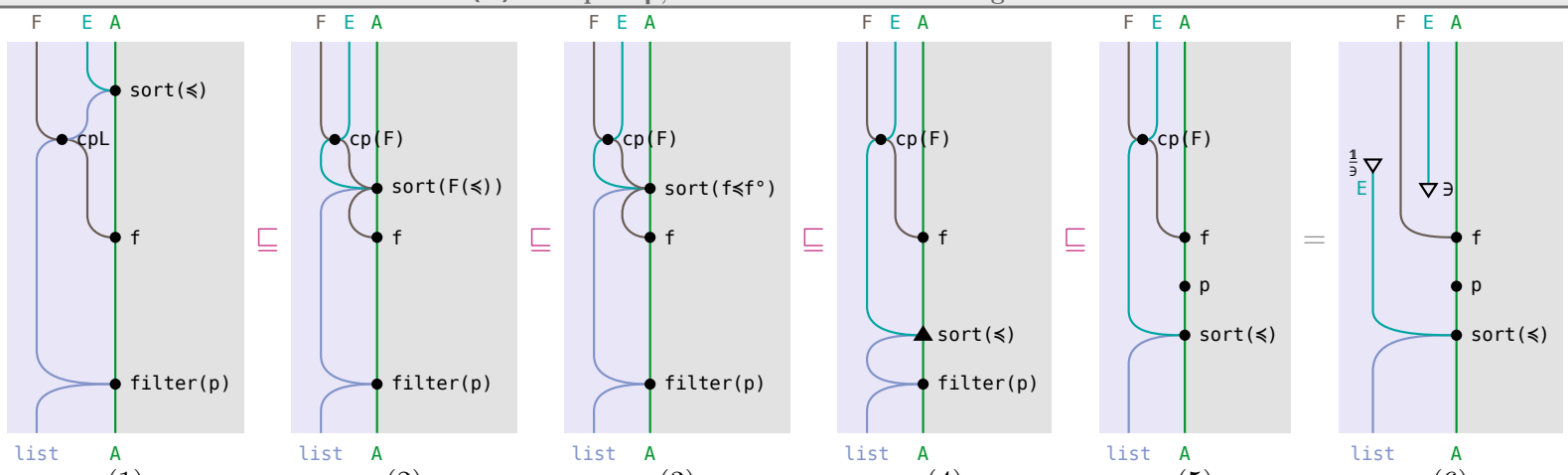
every list got by sorting the set and then thinning it by `Q` is also got by thinning the set by `Q` and then sorting.



1	`sort(⩽) thinL(Q)`	`sort(⩽) ≜ setify° ordered(⩽)`
2	`≜ setify° ordered thinL(Q)`	`ordered thinL(Q) ⊑ thinL(Q) ordered`
3	`⊑ setify° thinL(Q) ordered`	`setify° thinL(Q) ⊑ thin(Q) setify°`
4	`⊑ thin(Q) setify° ordered`	
5	`= thin(Q) sort(⩽)`	

Coreflexive(p); f Pres ⩽
⇒ F(sort(⩽)) cpL f′ filter(p) ⊑ $\frac{\mathsf{F(∋)\ f\ p}}{\mathsf{∋}}$ sort(⩽) Lemma 8.1

sorting each argument set (**F(sort(⩽))**), forming every combination of their elements (**cpL**), applying **f** to each (**f′**) and keeping those that pass **p** (**filter(p)**) gives one of the sorted lists of the set of all **f(u)** that pass **p**, with **u** drawn from the argument sets.



1	F(sort(⩽)) cpL f′ filter(p)	F(sort(⩽)) cpL ⊑ cp(F) sort(F(⩽))
2	⊑ cp(F) sort(F(⩽)) f′ filter(p)	Map(f) ⇒ f Pres ⩽ ⇔ F(⩽) ⊑ f ⩽ f°
3	⊑ cp(F) sort(f ⩽ f°) f′ filter(p)	sort(f ⩽ f°) f′ ⊑ P(f) sort(⩽)
4	⊑ cp(F) P(f) sort(⩽) filter(p)	Coreflexive(p) ⇒ sort(⩽) filter(p) ⊑ E(p) sort(⩽)
5	⊑ cp(F) P(f) E(p) sort(⩽)	Map(f) ⇒ P(f) = E(f)
6	= $\frac{\mathsf{F(∋)\ f\ p}}{\mathsf{∋}}$ sort(⩽)	

(8.3.1b)

At F(A,X)=1+A×X, f(r(a,x))=a+x, ⩽ is ≤, p passes the odd numbers, input r(3,{4,1,2}).

	left side	what it does	result
1	input	an r(a,xs), xs a set	r(3,{4,1,2})
2	F(sort(⩽))	sorts the set in the X place; a is untouched	r(3,[1,2,4])
3	cpL	pairs a with each element, in list order	[r(3,1),r(3,2),r(3,4)]
4	f′	f on each	[4,5,7]
5	filter(p)	keeps the odd ones, order kept	[5,7]

	right side	what it does	result
1	input		r(3,{4,1,2})
2	F(∋)	relates r(3,xs) to each r(3,x) with x∈xs	r(3,4), r(3,1), r(3,2)
3	f		7, 4, 5
4	p	only odd values pass	7, 5
5	Λ(…)	collects every result into one set	{5,7}
6	sort(⩽)		[5,7]

- ⊑: the left's [5,7] is one result of the right; with ties sort(⩽) has several orders.
- f preserving ⩽ is used at f′: [1,2,4] is sorted, so [4,5,7] is; filter(p) only drops elements. With f(r(a,x))=a−x the left gives [1,−1] after p, not sorted, so ⊑ fails.

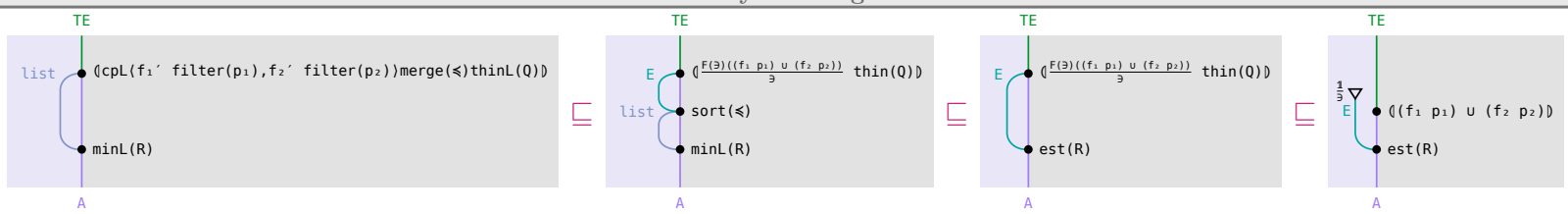
§8.3.2. Binary thinning

(8.3.2a)

Binary thinning data: Q a preorder with Q⊑R and both f_1p_1, f_2p_2 monotonic on Q; ⩽ a connected preorder with both f_1, f_2 monotonic on ⩽; R a preorder.

Coreflexive(p₁); Coreflexive(p₂); Q ⊑ R; preorder(Q); preorder(R); (f₁ p₁) Pres Q; (f₂ p₂) Pres Q; preorder(⩽); connected(⩽); f₁ Pres ⩽; f₂ Pres ⩽
⇒ ⦇cpL ⟨f₁′ filter(p₁),f₂′ filter(p₂)⟩ merge(⩽) thinL(Q)⦈ minL(R) ⊑ ⦇(f₁ p₁) ∪ (f₂ p₂)⦈_∋ est(R) Theorem 8.2

a fold on sorted lists of partial solutions, thinned at every step, refines the thinning specification — at binary thinning data.



(1) (2) (3) (4)

1	⦇cpL ⟨f₁′ filter(p₁),f₂′ filter(p₂)⟩ merge(⩽) thinL(Q)⦈ minL(R)	(8.3.2c) Coreflexive(p₁); Coreflexive(p₂); preorder(⩽); connected(⩽); f₁ Pres ⩽; f₂ Pres ⩽ ⇒ F(sort(⩽)) cpL ⟨f₁′ filter(p₁),f₂′ filter(p₂)⟩ merge(⩽) thinL(Q) ⊑ ⦇F(∋) ((f₁ p₁) ∪ (f₂ p₂))⦈_∋ thin(Q) sort(⩽)
2	⊑ ⦇F(∋) ((f₁ p₁) ∪ (f₂ p₂))⦈_∋ thin(Q)⦈ sort(⩽) minL(R)	sort(⩽) minL(R) ⊑ est(R)
3	⊑ ⦇F(∋) ((f₁ p₁) ∪ (f₂ p₂))⦈_∋ thin(Q)⦈ est(R)	S₁ Pres Q; S₂ Pres Q ⇒ S₁ ∪ S₂ Pres Q
4	⊑ ⦇(f₁ p₁) ∪ (f₂ p₂)⦈_∋ est(R)	

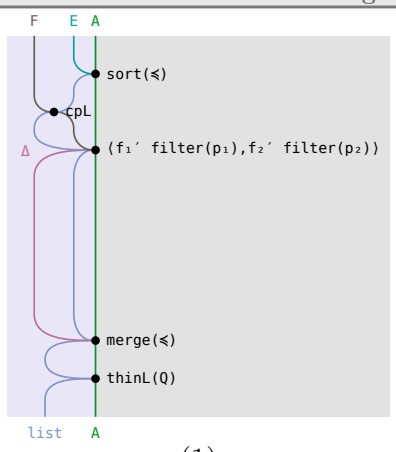
(8.3.2b)
At F(A,X)=1+A×X, f₁(l())=f₂(l())=0, f₁(r(a,x))=x, f₂(r(a,x))=a+x, p₁ passes all, p₂ passes values ≤6, ⩽ is ≤, Q is =, R is ≥, input [3,3,1].

	input	f₁′ filter(p₁)	f₂′ filter(p₂)	merge(⩽)	thinL(Q)
1	l()	[0]	[0]	[0,0]	[0]
2	r(1,[0])	[0]	[1]	[0,1]	[0,1]
3	r(3,[0,1])	[0,1]	[3,4]	[0,1,3,4]	[0,1,3,4]
4	r(3,[0,1,3,4])	[0,1,3,4]	[3,4,6]	[0,1,3,3,4,4,6]	[0,1,3,4,6]
5	minL(R)				6

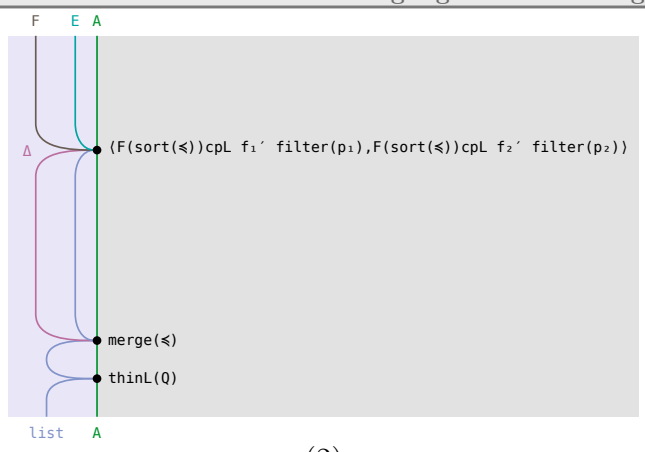
- merge(⩽) is one pass: both copies arrive ≤-sorted (fᵢ monotonic on ⩽, filter(pᵢ) only drops), so it only compares their two heads.
- thinL(Q) compares neighbours only; the list is ≤-sorted, so equal values are neighbours and one pass drops every repeat.

Coreflexive(p₁); Coreflexive(p₂); preorder(⩽); connected(⩽); f₁ Pres ⩽; f₂ Pres ⩽
⇒ F(sort(⩽)) cpL ⟨f₁′ filter(p₁),f₂′ filter(p₂)⟩ merge(⩽) thinL(Q) ⊑ $\frac{F(∋)\ ((f_1\ p_1)\ ∪\ (f_2\ p_2))}{∋}$ thin(Q) sort(⩽) (8.3.2c)

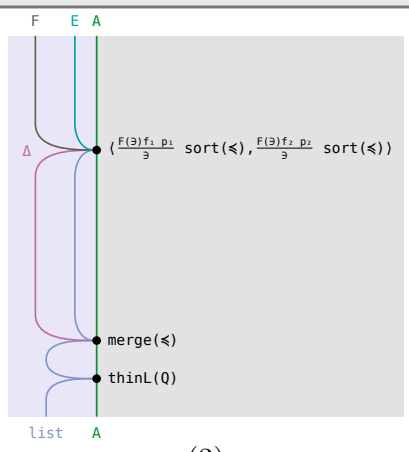
sorting the candidate set turns the thinning algebra into an algebra on sorted lists.



(1)

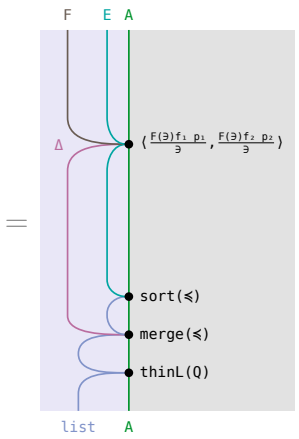


(2)

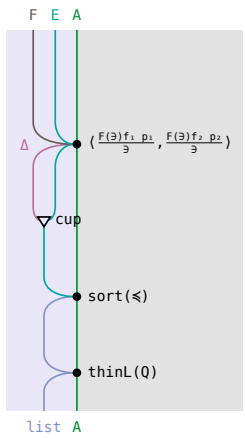


(3)

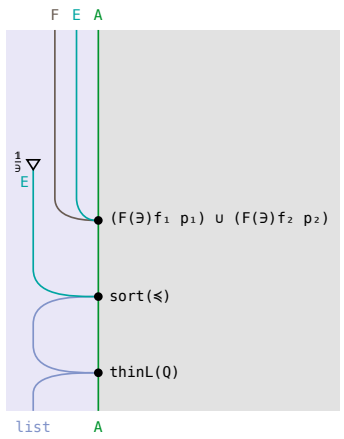
1	`F(sort(⩽)) cpL ⟨f₁′ filter(p₁),f₂′ filter(p₂)⟩ merge(⩽) thinL(Q)`	`X ⟨R,S⟩ ⊑ ⟨X R,X S⟩`
2	`⊑ (F(sort(⩽)) cpL f₁′ filter(p₁),F(sort(⩽)) cpL f₂′ filter(p₂)) merge(⩽) thinL(Q)`	Lemma 8.1 `Coreflexive(p); f Pres ⩽ ⇒ F(sort(⩽)) cpL f′ filter(p) ⊑ $\frac{F(∋)\ f\ p}{∋}$ sort(⩽)`
3	`⊑ ⟨$\frac{F(∋)\ f_1\ p_1}{∋}$ sort(⩽),$\frac{F(∋)\ f_2\ p_2}{∋}$ sort(⩽)⟩ merge(⩽) thinL(Q)`	`⟨X,Y⟩ (R × S) = ⟨X R,Y S⟩`



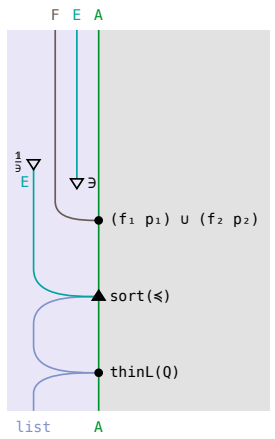
(4)



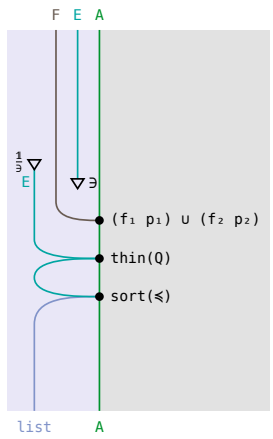
(5)



(6)



(7)



(8)

4	`= ⟨$\frac{F(∋)\ f_1\ p_1}{∋}$,$\frac{F(∋)\ f_2\ p_2}{∋}$⟩ (sort(⩽) × sort(⩽)) merge(⩽) thinL(Q)`	`preorder(⩽); connected(⩽) ⇒ (sort(⩽) × sort(⩽)) merge(⩽) ⊑ cup sort(⩽)`
5	`⊑ ⟨$\frac{F(∋)\ f_1\ p_1}{∋}$,$\frac{F(∋)\ f_2\ p_2}{∋}$⟩ cup sort(⩽) thinL(Q)`	`$\frac{R\ ∪\ S}{∋}$ = ⟨$\frac{R}{∋}$,$\frac{S}{∋}$⟩ cup`
6	`= $\frac{(F(∋)\ f_1\ p_1)\ ∪\ (F(∋)\ f_2\ p_2)}{∋}$ sort(⩽) thinL(Q)`	`R (S ∪ T) = (R S) ∪ (R T)`
7	`= $\frac{F(∋)\ ((f_1\ p_1)\ ∪\ (f_2\ p_2))}{∋}$ sort(⩽) thinL(Q)`	(8.6) `sort(⩽) thinL(Q) ⊑ thin(Q) sort(⩽)`
8	`⊑ $\frac{F(∋)\ ((f_1\ p_1)\ ∪\ (f_2\ p_2))}{∋}$ thin(Q) sort(⩽)`	

§8.4. The knapsack problem

(8.4a)

	definition	type	meaning
1	FX = L + E × X	𝐴→𝐴	here L ≜ 1 and E ≜ Item: the trivial base, or an item and what is built from the rest
2	total(f) ≜ f′ sum	Int	add up f over the items of the list
3	subseq ≜ ⦇[[],cons ∪ π₂]⦈	[A]→[A]	every subsequence of the list
4	value ≜ val′ sum	[Item]→Int	the total value of a packing
5	weight ≜ wt′ sum	[Item]→Int	the total weight of a packing
6	[nil,cons] = [[],cons]	F[Item]→ [Item]	keep the item: the list constructor
7	within(w,xs) ⇔ weight(xs) ≤ w	[Item]→[Item]	the packing weighs at most w
8	S ≜ ([nil,cons] within(w)) ∪ [nil,π₂]	Alg(F,[Item])	at each item, keep it if the packing still fits, or drop it
9	R ≜ value ≥ value°	[Item]→[Item]	x is worth at least as much as y
10	Q(val,wt) ≜ R ∩ (weight ≤ weight°)	[Item]→[Item]	x is worth at least as much as y and weighs no more

(8.4b)

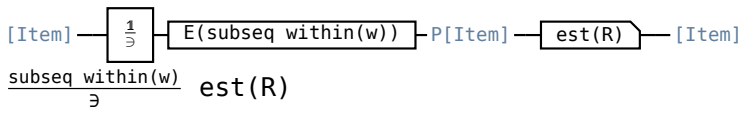
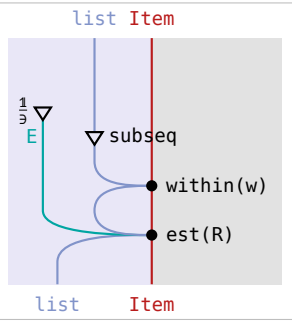
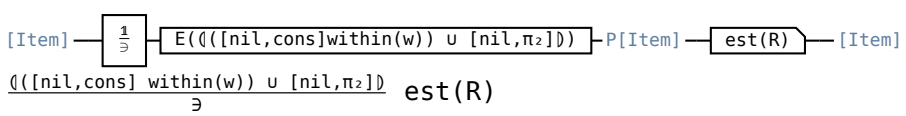
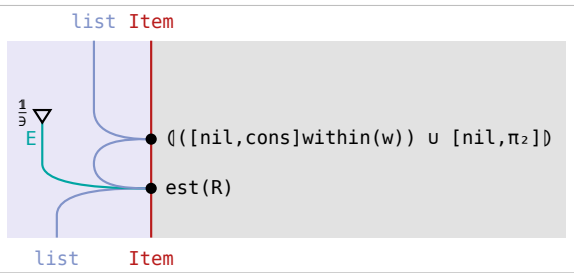
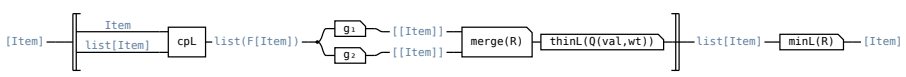
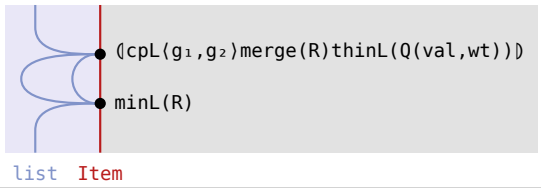
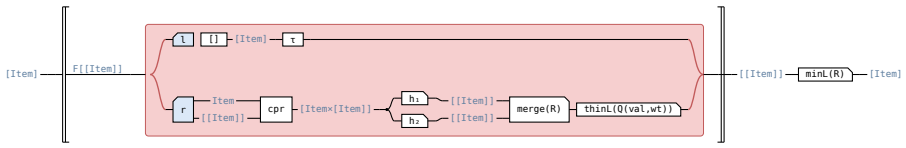
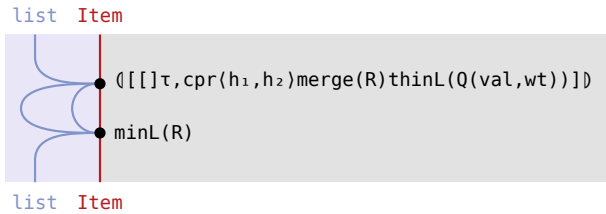
	the law	what it says
1	(𝟙×R) (cons (within w))⊑cons (within w)R FALSE	not monotonic on R: a selection of greater value need not still fit once one more item goes in
2	([nil,cons] within(w)) Pres Q(val,wt) [nil,π₂] Pres Q(val,wt) ,	both halves are monotonic on Q once ties in value are broken by weight
3	[nil,cons] Pres R [nil,π₂] Pres R	both algebras are monotonic on R itself, so R is the sort order P

§8.4.1. Derivation

0 ≤ w; (∀i. 0 ≤ wt(i)) (8.4.1a)

⇒ ⦇cpL ⟨g₁,g₂⟩ merge(R) thinL(Q(val,wt))⦈ minL(R) ⊑ $\frac{\text{subseq within(w)}}{\ni}$ est(R)

the knapsack problem, as a fold that thins the packings kept at each item

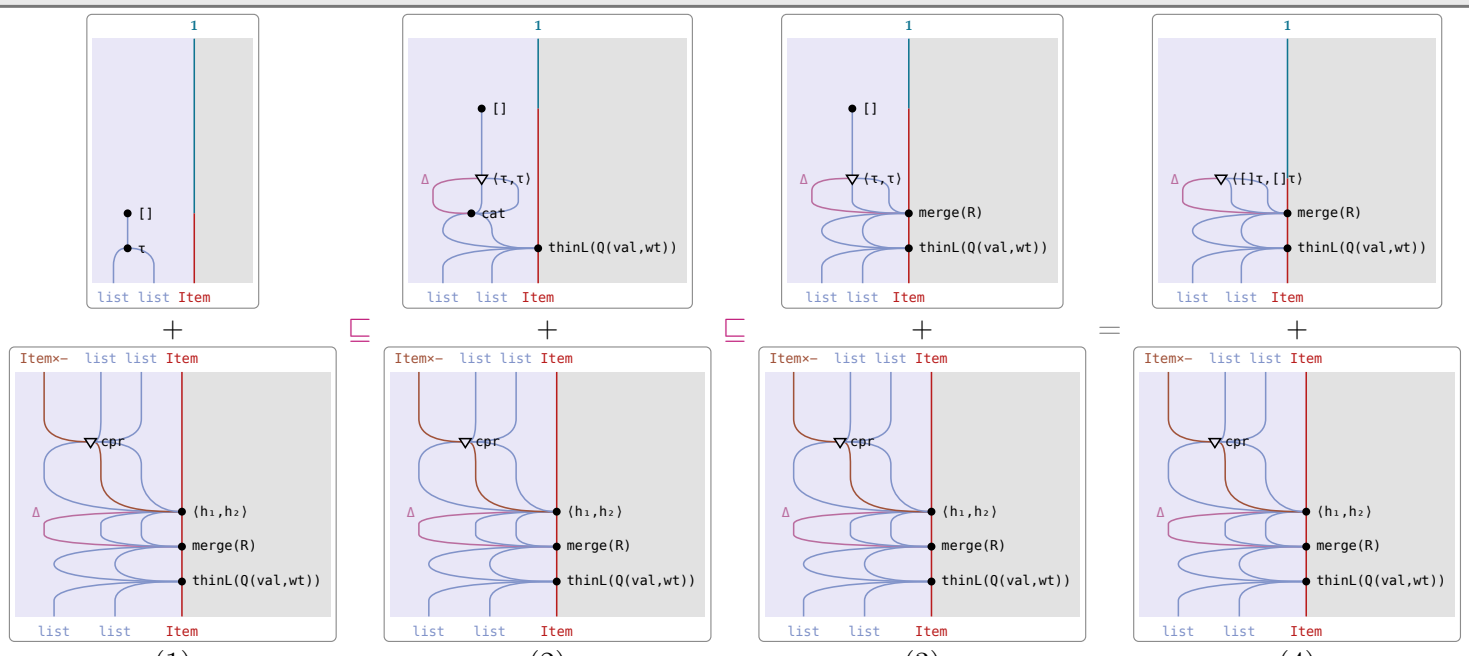
circuit	Hinze–Marsden
	
 = ⦇[nil,cons] within(w)) ∪ [nil,π₂]⦈ / ∋ est(R) fusion, weights non-negative.	
 ⊒ ⦇cpL ⟨g₁,g₂⟩ merge(R) thinL(Q(val,wt))⦈ minL(R) binary thinning theorem, at P≜R, F linear, Q from ([nil,cons] within(w)) Pres Q(val,wt) , [nil,π₂] Pres Q(val,wt)	
 ⊒ ⦇[[] τ,cpr ⟨h₁,h₂⟩ merge(R) thinL(Q(val,wt))]⦈ minL(R) 0 ≤ w ⇒ [[] τ,cpr ⟨h₁,h₂⟩ merge(R) thinL(Q(val,wt))] ⊑ cpL ⟨g₁,g₂⟩ merge(R) thinL(Q(val,wt)) under the fold; minL(R) is head, packings coming out in descending value	

§8.4.2. Splitting the fold's body

0 ≤ w (8.4.2a)

⇒ [[] τ,cpr ⟨h₁,h₂⟩ merge(R) thinL(Q(val,wt))] ⊑ cpL ⟨g₁,g₂⟩ merge(R) thinL(Q(val,wt))

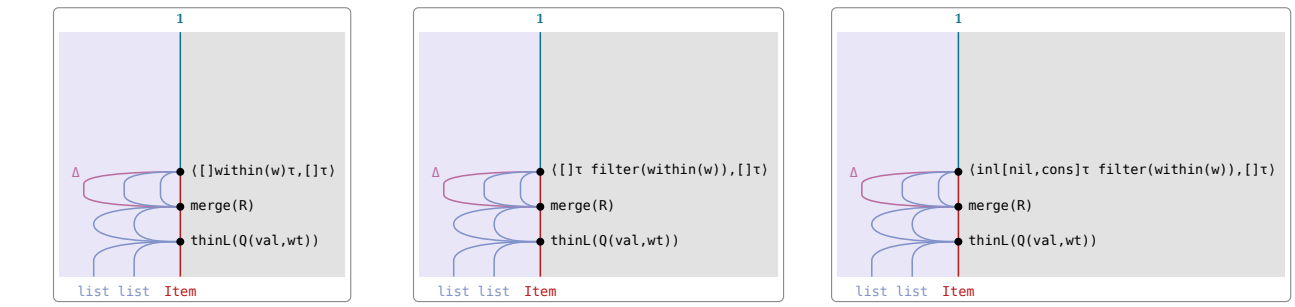
the fold's body splits into a base case **nil τ** and a step that extends each kept packing by the item or leaves it out



1	[[] τ,cpr ⟨h₁,h₂⟩ merge(R) thinL(Q(val,wt))]	1 ⊑ Q(val,wt)
2	⊑ [[] ⟨τ,τ⟩ cat thinL(Q(val,wt)),cpr ⟨h₁,h₂⟩ merge(R) thinL(Q(val,wt))]	1 ⊑ R
3	⊑ [[] ⟨τ,τ⟩ merge(R) thinL(Q(val,wt)),cpr ⟨h₁,h₂⟩ merge(R) thinL(Q(val,wt))]	Map(f) ⇒ f ⟨τ,τ⟩ = ⟨f τ,f τ⟩

4 = [([] τ,[] τ) merge(R) thinL(Q(val,wt)),cpr ⟨h₁,h₂⟩ merge(R) thinL(Q(val,wt))]

0 ≤ w
⇒ [] within(w) = []



(5) (6) (7)

5 = [(⟨[] within(w) τ,[] τ⟩ merge(R) thinL(Q(val,wt)),cpr ⟨h₁,h₂⟩ merge(R) thinL(Q(val,wt))⟩]

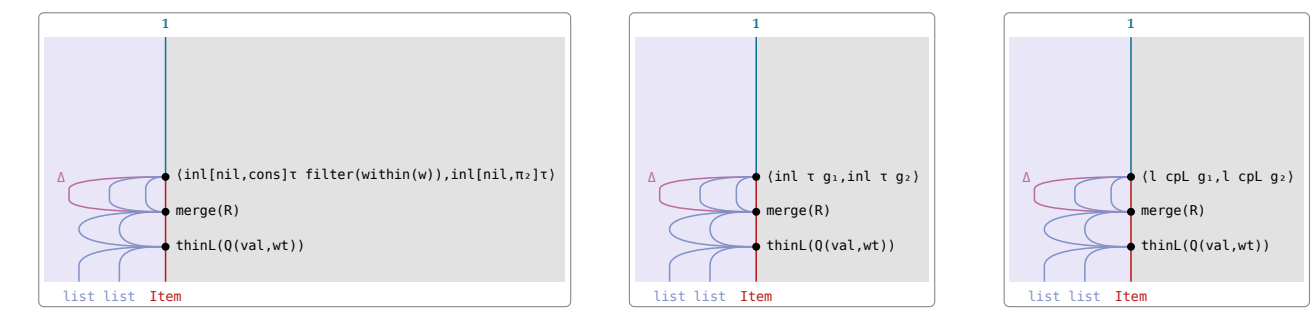
Coreflexive(within(w))

6 ⊑ [([] τ filter(within(w)),[] τ) merge(R) thinL(Q(val,wt)),cpr ⟨h₁,h₂⟩ merge(R) thinL(Q(val,wt))]

inl [nil,cons] = []

7 = [(inl [nil,cons] τ filter(within(w)),[] τ) merge(R) thinL(Q(val,wt)),cpr ⟨h₁,h₂⟩ merge(R) thinL(Q(val,wt))]

inl [nil,π₂] = []



(8) (9) (10)

8 = [(inl [nil,cons] τ filter(within(w)),inl [nil,π₂] τ) merge(R) thinL(Q(val,wt)),cpr (h₁,h₂) merge(R) thinL(Q(val,wt))]

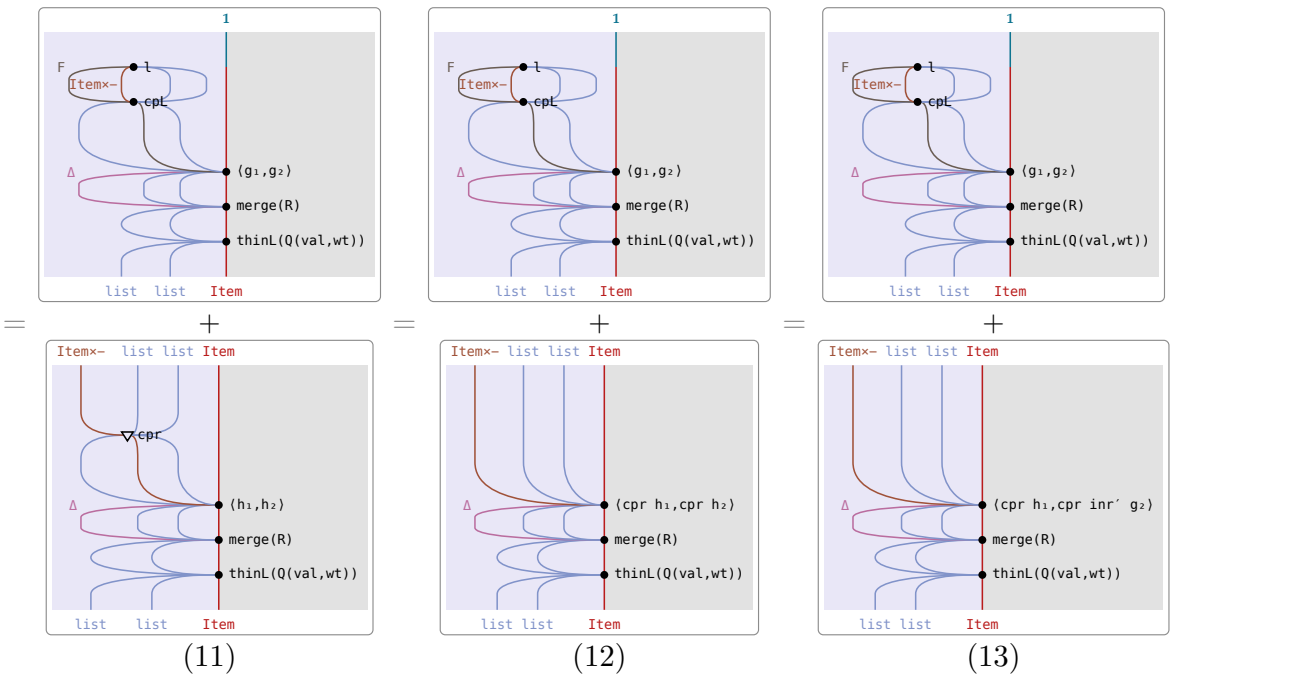
τ R′ = R τ

9 = [(inl τ g₁,inl τ g₂) merge(R) thinL(Q(val,wt)),cpr ⟨h₁,h₂⟩ merge(R) thinL(Q(val,wt))]

l cpL = inl τ

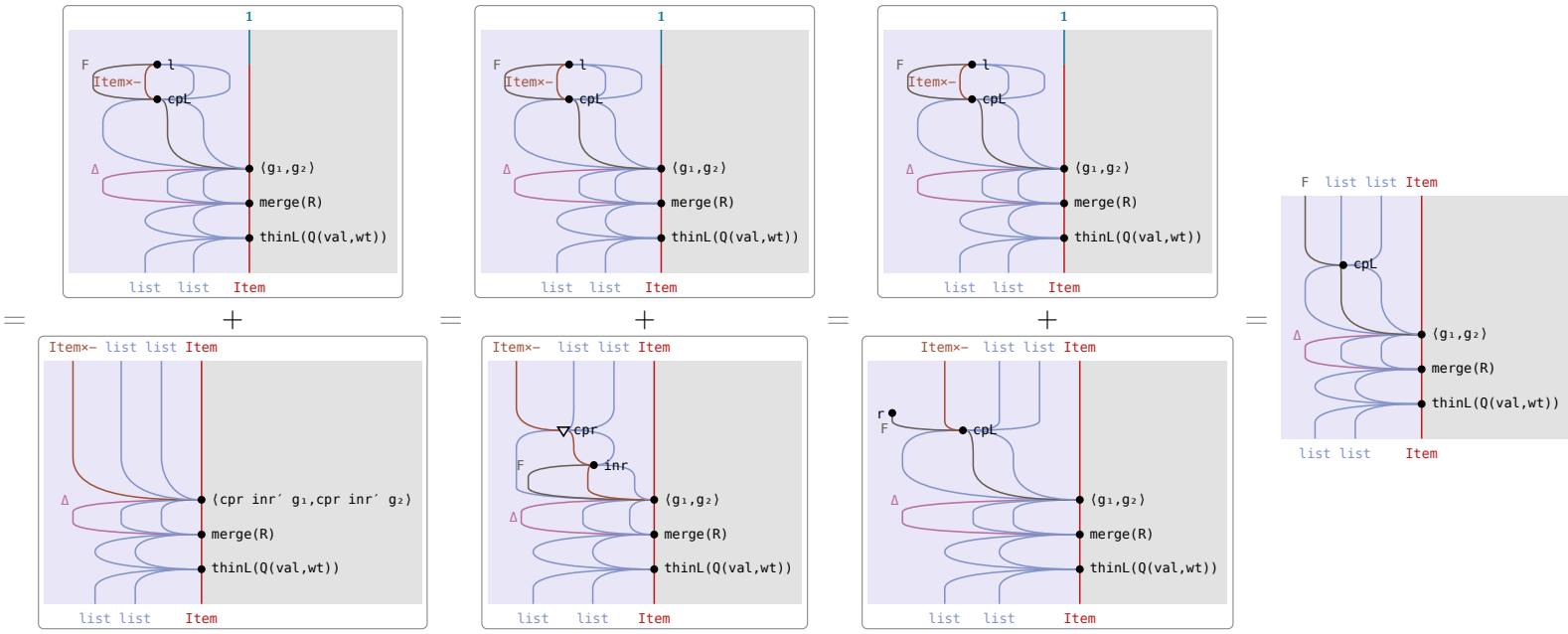
10 = [(l cpL g₁,l cpL g₂) merge(R) thinL(Q(val,wt)),cpr ⟨h₁,h₂⟩ merge(R) thinL(Q(val,wt))]

Map(f)
⇒ f ⟨R,S⟩ = ⟨f R,f S⟩



(11) (12) (13)

```
= [l cpL ⟨g₁,g₂⟩ merge(R) thinL(Q(val,wt)),cpr ⟨h₁,h₂⟩ merge(R) thinL(Q(val,wt))]   Map(cpr)
                                                                                   ⇒ cpr ⟨h₁,h₂⟩ = ⟨cpr h₁,cpr h₂⟩
= [l cpL ⟨g₁,g₂⟩ merge(R) thinL(Q(val,wt)),⟨cpr h₁,cpr h₂⟩ merge(R) thinL(Q(val,wt))]   inr′ [nil,π₂]′ = π₂′
= [l cpL ⟨g₁,g₂⟩ merge(R) thinL(Q(val,wt)),⟨cpr h₁,cpr inr′ g₂⟩ merge(R) thinL(Q(val,wt))]   inr′ [nil,cons]′ = cons′
```




```
= [l cpL ⟨g₁,g₂⟩ merge(R) thinL(Q(val,wt)),⟨cpr inr′ g₁,cpr inr′ g₂⟩ merge(R) thinL(Q(val,wt))]   Map(f)
                                                                                                ⇒ f ⟨R,S⟩ = ⟨f R,f S⟩
= [l cpL ⟨g₁,g₂⟩ merge(R) thinL(Q(val,wt)),cpr inr′ ⟨g₁,g₂⟩ merge(R) thinL(Q(val,wt))]   r cpL = cpr inr′
= [l cpL ⟨g₁,g₂⟩ merge(R) thinL(Q(val,wt)),r cpL ⟨g₁,g₂⟩ merge(R) thinL(Q(val,wt))]   l T = R; r T = S
                                                                                      ⇒ T = [R,S]
= cpL ⟨g₁,g₂⟩ merge(R) thinL(Q(val,wt))
```

(8.4.2b)

stage	type	value	meaning
input	`[Int×Int]`	`[(6,5), (5, 4), (4, 6), (3, 3)]`	the items `(value, weight)`, left to right
`kept(val,wt)(w)`	`[[Int×Int]]`	`[[(5, 4), (3, 3)], [(6, 5), (3, 3)], [(3, 3)], [(6, 5), (5, 4)], [(5, 4)], [(6, 5)], []]`	the fold keeps seven packings: the thinning drops one less valuable and heavier than another
`knap(val,wt)(w,xs)`	`[[Int×Int]]`	`[[(6, 5), (5, 4)]]`	the one of value `11` and weight `9`, `est(R)` of the kept packings

§8.5. The paragraph problem

(8.5a)

	definition	type	meaning
1	FX = L + E × X	𝐴→𝐴	here L ≜ Word and E ≜ Word: one word, or a word and what is built from the rest
2	Line ≜ L(Word)	Type	a line is a non-empty list of words
3	Para ≜ L(Line)	Type	a paragraph is a non-empty list of lines
4	μF ::= wrap(L) | cons(E,μF)	L→μF	the one-element non-empty list holding a
5	new(a,xss) ≜ [[a]]⧺xss	Para	start a new first line holding only the word a
6	glue(a,[xs]) = [[a]⧺xs] glue(a,[xs]⧺a') = [[a]⧺xs]⧺a'	Para→ Para	put the word a at the front of the first line
7	partition ≜ ⦇[τ²,new] ∪ [τ²,glue]⦈	L(Word)→ Para	every way of breaking the words into lines
8	width[a] = len(a) width([a]⧺xs) = (len(a) + width(xs)) + 1	Line→Int	the lengths of the words plus one space between neighbours
9	head[xs] = xs head([xs]⧺a) = xs	Para→ Line	the first line of the paragraph
10	fits(w,[xs]) = width(xs) ≤ w fits(w,[xs]⧺a) = width(xs) ≤ w ∧ fits(w,a)	Para→ Para	every line is at most w wide
11	ok(w,xss) ≜ width(head(xss)) ≤ w	Para→ Para	the paragraphs whose first line is at most w wide
12	sqr(n) ≜ nn	Int	n squared
13	waste(w,[xs]) = 0 waste(w,[xs]⧺a) = sqr(w−width(xs)) + waste(w,a)	Para→Int	the squared white space left on every line but the last, added up
14	R ≜ waste(w) ≤ waste(w)°	Para→ Para	p wastes no more than q
15	Q(w) ≜ R ∩ (head head°)	Para→ Para	p wastes no more than q and has the same first line
16	start(a) ≜ [[[a]]]	[Para]	the one candidate for a single word: the paragraph of one line holding that word

(8.5b)

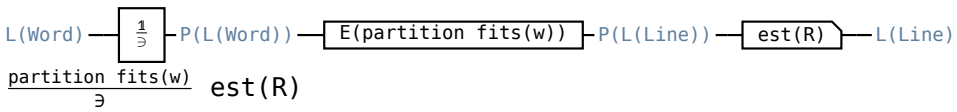
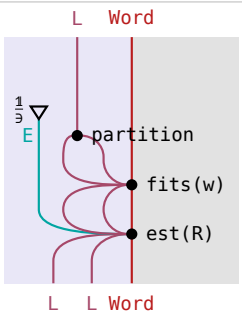
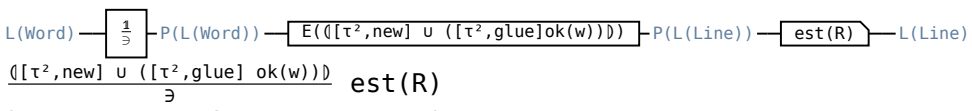
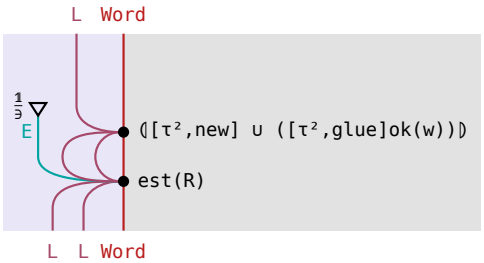
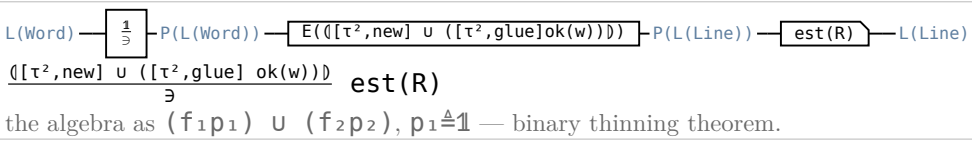
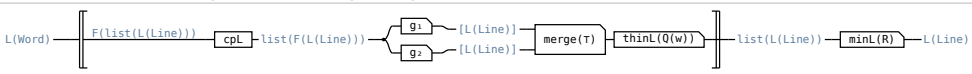
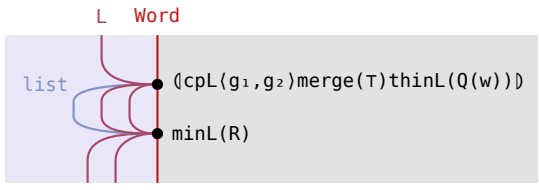
	the law	what it says
1	(𝟙×R) glue⊑glue R FALSE	glue is not monotonic on R: the waste of a paragraph depends on its whole first line, so no greedy algorithm solves this
2	[τ²,new] Pres Q(w) (∀a. 0 ≤ len(a)) ⇒ ([τ²,glue] ok(w)) Pres Q(w) cons monotonic on collect⊑collect°. ,	both halves are monotonic on Q once ties in waste are broken by the first line
3	merge(⊤) = cat	⊤ needs no sorting at all
4	[τ²,new] Pres ⊤ [τ²,glue] Pres ⊤	both algebras are monotonic on ⊤, so ⊤ is the sort order P

§8.5.1. Derivation

(∀a. 0 ≤ len(a)); (∀a. len(a) ≤ w) (8.5.1a)

⇒ ⦇cpL ⟨g₁,g₂⟩ merge(⊤) thinL(Q(w))⦈ minL(R) ⊑ $\frac{\text{partition fits(w)}}{\ni}$ est(R)

a paragraph laid out as a fold that thins the layouts kept at each word

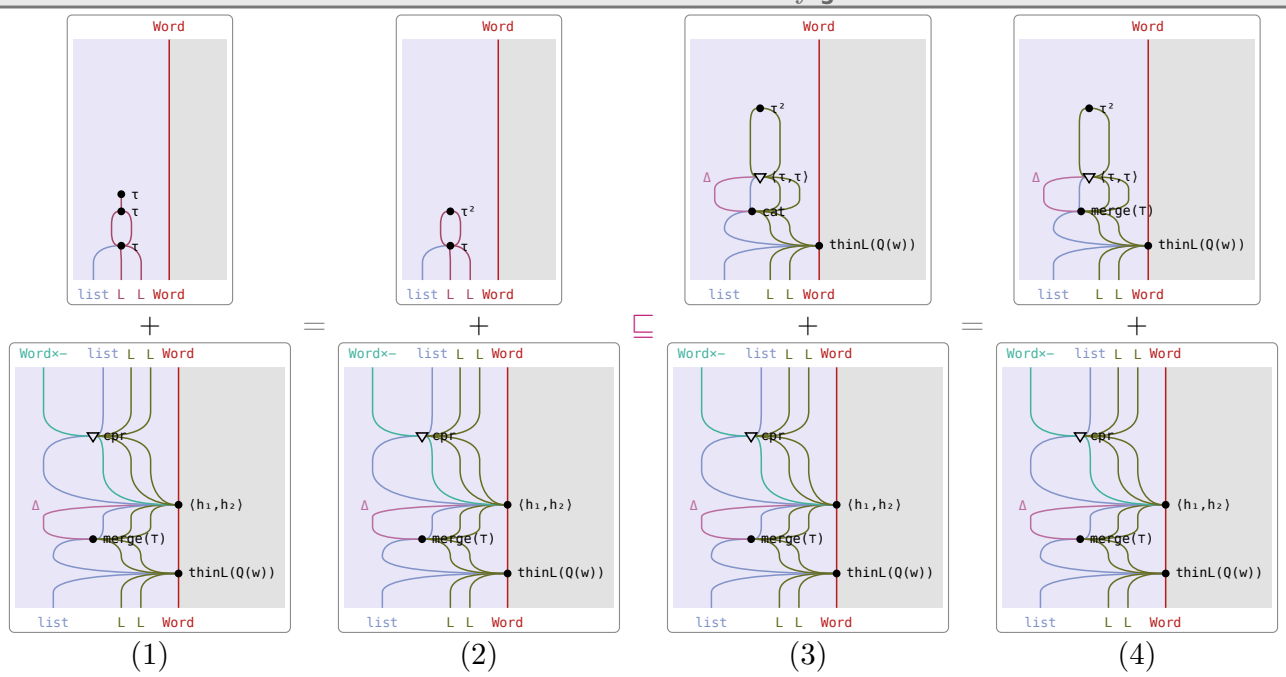
circuit	Hinze–Marsden
 $\frac{\text{partition fits(w)}}{\ni}$ est(R)	
=  $\frac{\text{⦇[τ²,new] ∪ ([τ²,glue] ok(w))⦈}}{\ni}$ est(R) fusion, every word fits on a line by itself.	
=  $\frac{\text{⦇[τ²,new] ∪ ([τ²,glue] ok(w))⦈}}{\ni}$ est(R) the algebra as (f₁p₁) ∪ (f₂p₂), p₁≜𝟙 — binary thinning theorem.	
⊒  ⦇cpL ⟨g₁,g₂⟩ merge(⊤) thinL(Q(w))⦈ minL(R) binary thinning theorem, at P≜⊤ with merge ⊤=cat (merge(⊤) = cat), Q from [τ²,new] Pres Q(w) , (∀a. 0 ≤ len(a)) ⇒ ([τ²,glue] ok(w)) Pres Q(w)	
= ⦇[start,cpr ⟨h₁,h₂⟩ cat thinL(Q)]⦈ minL(R) cpL=τ+cpr, gᵢ along the coproduct — start(a) ≜ [[[a]]] , new(a,xss) ≜ [[a]]#xss , glue(a,[xs]) = [[a]#xs], glue(a,[xs]#a') = [[a]#xs]#a'	

§8.5.2. Splitting the fold's body

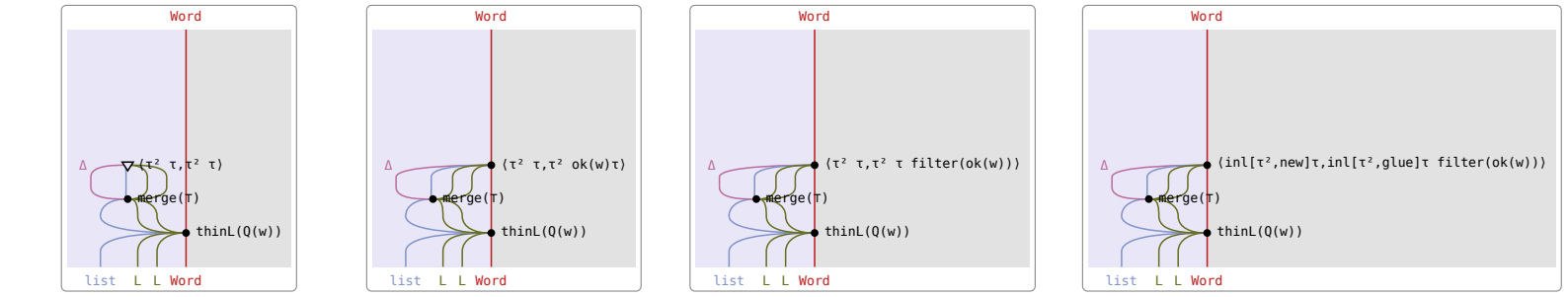
(∀a. len(a) ≤ w) (8.5.2a)

⇒ [τ² τ,cpr ⟨h₁,h₂⟩ merge(⊤) thinL(Q(w))] ⊑ cpL ⟨g₁,g₂⟩ merge(⊤) thinL(Q(w))

the fold's body splits into a base case **start** for a single word and a step that extends each kept layout by **new** and by **glue**

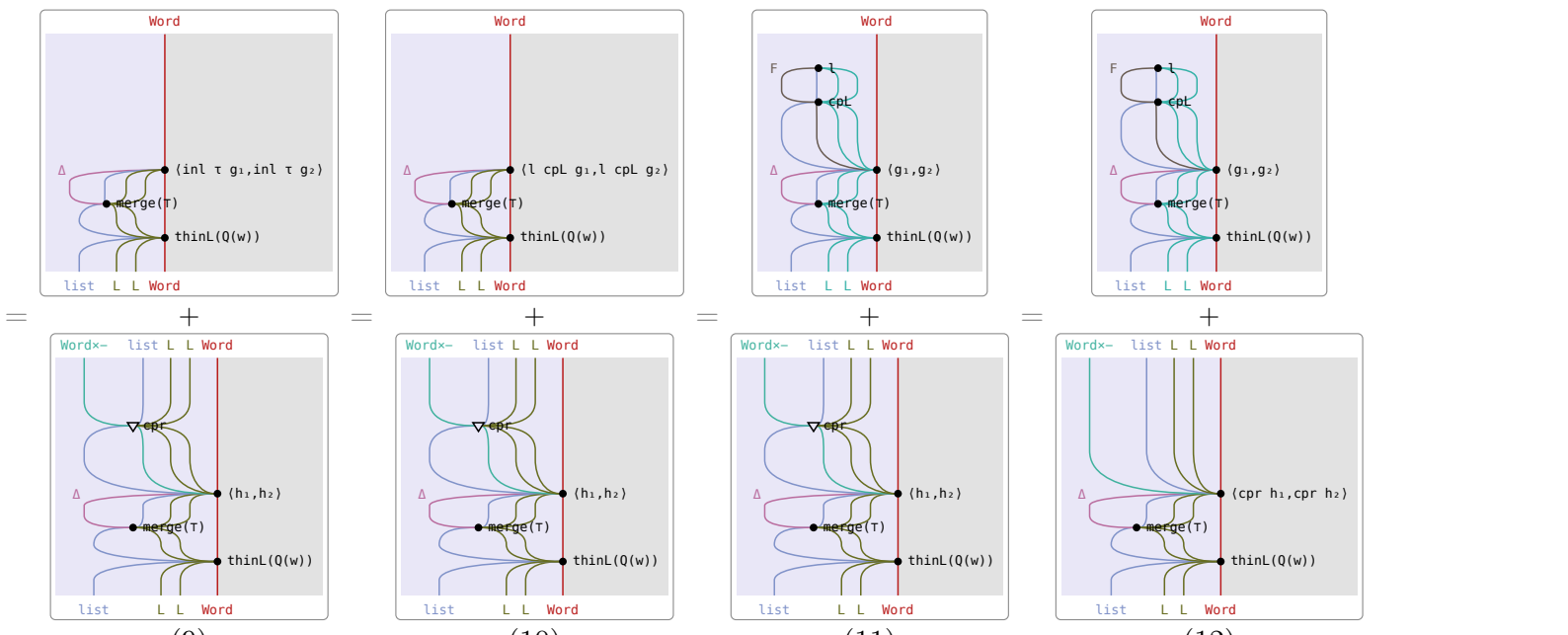


1	[τ² τ,cpr ⟨h₁,h₂⟩ merge(⊤) thinL(Q(w))]	τ² = τ²
2	= [τ² τ,cpr ⟨h₁,h₂⟩ merge(⊤) thinL(Q(w))]	𝟙 ⊑ Q(w)
3	⊑ [τ² ⟨τ,τ⟩ cat thinL(Q(w)),cpr ⟨h₁,h₂⟩ merge(⊤) thinL(Q(w))]	merge(⊤) = cat
4	= [τ² ⟨τ,τ⟩ merge(⊤) thinL(Q(w)),cpr ⟨h₁,h₂⟩ merge(⊤) thinL(Q(w))]	Map(f) ⇒ f ⟨τ,τ⟩ = ⟨f τ,f τ⟩



(5) (6) (7) (8)

5	= [⟨τ² τ,τ² τ⟩ merge(T) thinL(Q(w)),cpr ⟨h₁,h₂⟩ merge(T) thinL(Q(w))]	(∀a. len(a) ≤ w) ⇒ τ² ok(w) = τ²
6	= [⟨τ² τ,τ² ok(w) τ⟩ merge(T) thinL(Q(w)),cpr ⟨h₁,h₂⟩ merge(T) thinL(Q(w))]	Coreflexive(ok(w))
7	⊑ [⟨τ² τ,τ² τ filter(ok(w))⟩ merge(T) thinL(Q(w)),cpr ⟨h₁,h₂⟩ merge(T) thinL(Q(w))]	inl [g,h] = g
8	= [⟨inl [τ²,new] τ,inl [τ²,glue] τ filter(ok(w))⟩ merge(T) thinL(Q(w)),cpr ⟨h₁,h₂⟩ merge(T) thinL(Q(w))]	τ R′ = R τ



(9) (10) (11) (12)

9	= [⟨inl τ g₁,inl τ g₂⟩ merge(T) thinL(Q(w)),cpr ⟨h₁,h₂⟩ merge(T) thinL(Q(w))]	l cpL = inl τ
10	= [⟨l cpL g₁,l cpL g₂⟩ merge(T) thinL(Q(w)),cpr ⟨h₁,h₂⟩ merge(T) thinL(Q(w))]	Map(f) ⇒ f ⟨R,S⟩ = ⟨f R,f S⟩
11	= [l cpL ⟨g₁,g₂⟩ merge(T) thinL(Q(w)),cpr ⟨h₁,h₂⟩ merge(T) thinL(Q(w))]	Map(cpr) ⇒ cpr ⟨h₁,h₂⟩ = ⟨cpr h₁,cpr h₂⟩
12	= [l cpL ⟨g₁,g₂⟩ merge(T) thinL(Q(w)),⟨cpr h₁,cpr h₂⟩ merge(T) thinL(Q(w))]	inr′ [g,h]′ = h′

```
      (13)                                                                    (14)                                                                    (15)                                                                    (16)
13    = [l cpL ⟨g₁,g₂⟩ merge(T) thinL(Q(w)),⟨cpr inr′ g₁,cpr inr′ g₂⟩ merge(T) thinL(Q(w))]   Map(f)
                                                                                                                                                      ⇒ f ⟨R,S⟩ = ⟨f R,f S⟩
14    = [l cpL ⟨g₁,g₂⟩ merge(T) thinL(Q(w)),cpr inr′ ⟨g₁,g₂⟩ merge(T) thinL(Q(w))]          r cpL = cpr inr′
15    = [l cpL ⟨g₁,g₂⟩ merge(T) thinL(Q(w)),r cpL ⟨g₁,g₂⟩ merge(T) thinL(Q(w))]          l T = R; r T = S
                                                                                                                                                      ⇒ T = [R,S]
16    = cpL ⟨g₁,g₂⟩ merge(T) thinL(Q(w))
```

(8.5.2b)

	stage	type	value	meaning
1	input	`L(Int)`	`[1, 2, 1, 3]`	the words' lengths, left to right
2	`kept(w)`	`[L (L Int)]`	`[[[1], [2, 1], [3]], [[1, 2], [1, 3]]]`	the fold keeps two layouts: the thinning drops `[[2],[1],[3]]` for `[[2],[1,3]]` and `[[1],[2],[1,3]]` for `[[1],[2,1],[3]]`, each with the same first line and less waste
3	`paragraph(w,xs)`	`[L (L Int)]`	`[[[1, 2], [1, 3]]]`	the one of waste `1`, `est(R)` of the kept layouts

§8.6. Bitonic tours

(8.6a)

	definition	type	meaning
1	FX = L + E × X	𝒜→𝒜	here L ≜ City×City and E ≜ City: the last two cities, or a city and what is built from the rest
2	Journey(City) ≜ μF	Type	a list of at least two cities, the last two held as a pair
3	Tour(City) ≜ Journey(City) × Journey(City)	Type	the outward journey and the return journey
4	head[a, b] = a head([a]⧺a') = a	Journey(City)→ City	the first city of the journey
5	next[a, b] = b next([a]⧺a') = head(a')	Journey(City)→ City	the second city of the journey
6	tail([a, b]) = [b] tail([a]⧺a') = a'	Journey(City)→ L(City)	the journey without its first city, a non-empty list
7	[a]⧺[c] = [a, c] [a]⧺[b]⧺xs = [a]⧺[b]⧺xs	L(City)→ Journey(City)	a put in front of a non-empty list, a journey
8	replaceHead = (𝟙 × tail) ⧺	Journey(City)→ Journey(City)	replace the first city of the journey by a
9	outcost[a, b] = tc(a,b) outcost([a]⧺a') = tc(a,head(a')) + outcost(a')	Journey(City)→ Int	the cost of travelling the journey forwards
10	incost[a, b] = tc(b,a) incost([a]⧺a') = tc(head(a'),a) + incost(a')	Journey(City)→ Int	the cost of travelling the journey backwards
11	cost(t) ≜ outcost(π₁(t)) + incost(π₂(t))	Int	the outward journey forwards plus the return journey backwards
12	start(a,b) = ([a, b],[a, b])	City×City→ Tour(City)	the tour of the last two cities, both journeys being that one edge
13	dropl(a,t) ≜ (replaceHead(a,π₁(t)),[a]⧺π₂(t))	Tour(City)	a replaces the first city of the outward journey and is put in front of the return
14	dropr(a,t) ≜ ([a]⧺π₁(t),replaceHead(a,π₂(t)))	Tour(City)	a is put in front of the outward journey and replaces the first city of the return
15	[start,dropl](l(p)) = start(p) [start,dropl](r(q)) = dropl(π₁(q),π₂(q))	F(Tour(City))→ Tour(City)	start at the last two cities, then add each city by dropl
16	[start,dropr](l(p)) = start(p) [start,dropr](r(q)) = dropr(π₁(q),π₂(q))	F(Tour(City))→ Tour(City)	start at the last two cities, then add each city by dropr
17	tour ≜ ⦇[start,dropl] ∪ [start,dropr]⦈	μF→Tour(City)	every bitonic tour of the cities
18	R ≜ cost ≤ cost°	Tour(City)→ Tour(City)	t costs no more than t′
19	next₂(t) ≜ (next(π₁(t)),next(π₂(t)))	City×City	the second cities of both journeys
20	head₂(t) ≜ (head(π₁(t)),head(π₂(t)))	City×City	the first cities of both journeys
21	Q ≜ R ∩ (next₂ next₂°) ∩ (head₂ head₂°)	Tour(City)→ Tour(City)	t costs no more than t′, and both journeys of t and t′ share their first two cities

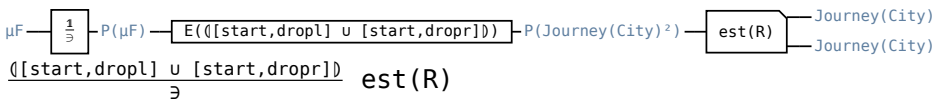
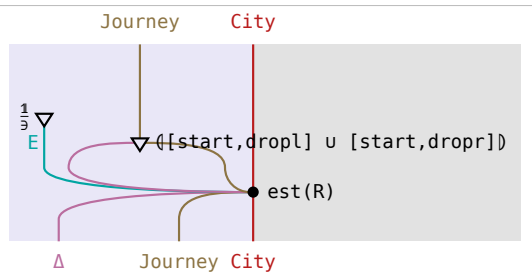
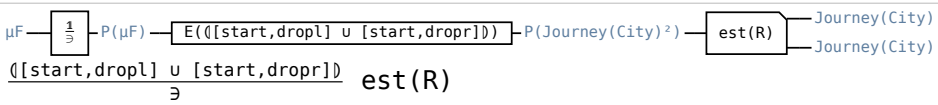
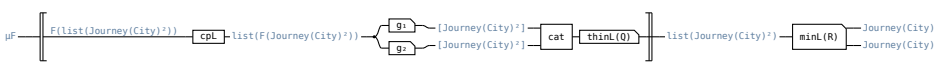
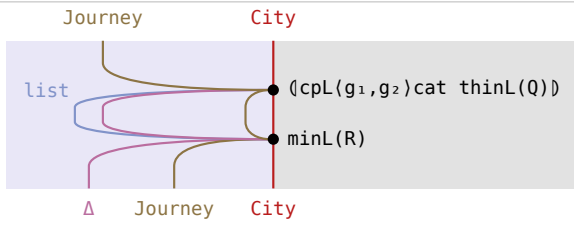
(8.6b)

	the law	what it says
1	(𝟙×R) dropl⊑dropl R FALSE (𝟙×R) dropr⊑dropr R FALSE	neither drop is monotonic on R: the two edges it adds and removes depend on head and next of both lists
2	[start,dropl] Pres Q [start,dropr] Pres Q ,	both are, once ties in cost are broken by the two second cities — the heads already agree among tours of one input
3	[start,dropl] Pres ⊤ [start,dropr] Pres ⊤	both algebras are monotonic on ⊤, so ⊤ is the sort order P

§8.6.1. Derivation

(⦇cpL ⟨g₁,g₂⟩ cat thinL(Q)⦈ minL(R) ⊑ ⦇[start,dropl] ∪ [start,dropr]⦈/∋ est(R)) (8.6.1a)

a least-cost bitonic tour, as a fold that thins the tours kept at each city

circuit	Hinze–Marsden
 ⦇[start,dropl] ∪ [start,dropr]⦈/∋ est(R)	
=  ⦇[start,dropl] ∪ [start,dropr]⦈/∋ est(R) tour ≜ ⦇[start,dropl] ∪ [start,dropr]⦈	
⊒  ⦇cpL ⟨g₁,g₂⟩ cat thinL(Q)⦈ minL(R) binary thinning theorem, at P≜T with merge T=cat, Q from [start,dropl] Pres Q , [start,dropr] Pres Q	
= ⦇[start τ,cpr ⟨dropl′,dropr′⟩ cat thinL(Q)]⦈ minL(R) cpL=τ+cpr, gᵢ=[start′,dropᵢ′] — start(a,b) = ([a, b],[a, b]) , dropl(a,t) ≜ (replaceHead(a,π₁(t)),[a]⧺π₂(t)) ; quadratic, two tours added per step	

(8.6.1b)

	stage	type	value	meaning
1	input	μF	[0, 3, 1, 4, 2]	the cities, left to right
2	kept	[μF×μF]	[([0, 2],[0, 3, 1, 4, 2]), ([0, 1, 2], [0, 3, 4, 2]), ([0, 3, 1, 2], [0, 4, 2]), ([0, 4, 2], [0, 3, 1, 2]), ([0, 3, 4, 2], [0, 1, 2]), ([0, 3, 1, 4, 2], [0, 2])]	the fold keeps six tours: the thinning drops the rest at each city
3	bitonic(xs)	[μF×μF]	[([0, 1, 2],[0, 3, 4, 2]), ([0, 3, 4, 2], [0, 1, 2])]	the two of cost 16, est(R) of the kept tours